Technical Reference Manual for the Pixel Array Control Monitor and Network (PACMAN) Card

Michael Mulhearn

November 12, 2026

Contents

1 Overview **3**

2 Hardware Implementation **4**
2.1 Package Pin to Schematic Netlist 4

3 Programmable Logic Firmware **6**
3.1 Clocks 8
3.2 Registers 11
3.3 Global Unit 14
3.4 TX Unit 15
3.5 RX Unit 21
3.6 Timing Unit 29
3.7 ADC Unit 30

4 PACMAN Server **30**
4.1 DAQ Interface 30

5 Developer Workflows **33**

6 Quality Assurance and Quality Control **34**
6.1 Feature Verification Overview 34
6.2 Requirement Verification 35

7 Hardware Checkout Procedure **35**
7.1 Bare Board 36
7.2 TRENZ Module and Linux System Checks 36
7.3 Linux-based Hardware Checkout Application 37
7.3.1 LED checkout 37
7.3.2 Board Power 38
7.3.3 VDDD and VDDA 38
7.3.4 POSI/PISO Loopback Testing with Random Numbers 39
7.3.5 Clock and Reset/Sync/Trigger Signals 40
7.3.6 Front Panel Checkout 41

A Specifications for the PACMAN Card **41**
A.1 Overview 41
A.2 Power 41
A.3 Digital Signals 42
A.4 Noise 42
A.5 Pixel Tile Interface 42
A.6 Front Panel Interface 43
A.7 Timing System Endpoint 43
A.8 Embedded Linux 43

B PACMAN Version History **44**

C Prototype Results **44**

1 Overview

The Pixel Array Control Monitor and Network (PACMAN) card is the warm electronics (located outside of the cryostat) for the pixellated liquid argon (LAr) time projection chamber (TPC) based on the LArPix ASIC. The PACMAN card provides power and digital I/O to arrays of LArPix ASICs hosted on pixel tile cards located within the cryostat. In the production system, each PACMAN card handles up to ten pixel tile cards. The PACMAN card provides analog test and monitor signals as well as digital control signals for clock, external trigger, and reset, and sync. The PACMAN card serves as a timing system end point, hosts a linux CPU, and is connected to the data acquisition system via ethernet.

The PACMAN card plays a supporting role in the LArPix system. It does not face technical challenges as severe as the cold electronics of the LArPix system and therefore a basic design constraint is that it in no way limits the performance of the supported system.

2 Hardware Implementation

2.1 Package Pin to Schematic Netlist

Netlist	Package Pin	Netlist	Package Pin
ADC_CLK	J15	SYNC[0]	D18
ADC_D[0]	F19	SYNC[1]	J17
ADC_D[10]	AA17	SYNC[2]	L17
ADC_D[11]	AB17	SYNC[3]	M17
ADC_D[1]	G19	SYNC[4]	N17
ADC_D[2]	H20	SYNC[5]	N18
ADC_D[3]	H19	SYNC[6]	J21
ADC_D[4]	R6	SYNC[7]	J22
ADC_D[5]	T6	SYNC[8]	J20
ADC_D[6]	U6	SYNC[9]	K21
ADC_D[7]	U5	SYNC_ISO	K18
ADC_D[8]	Y18	TEST_OUT[0]	AB2
ADC_D[9]	AA18	TEST_OUT[1]	AB1
ADC_EN	AB16	TILE_EN[0]	G21
ADC_OF	AA16	TILE_EN[1]	P18
ANALOG_PWR_EN	J16	TILE_EN[2]	P17
CDR_CLK_N	W18	TILE_EN[3]	K15
CDR_CLK_P	W17	TILE_EN[4]	L21
CDR_DATA_N	Y16	TILE_EN[5]	P21
CDR_DATA_P	W16	TILE_EN[6]	P20
CDR_LOL	V5	TILE_EN[7]	L22
CDR_LOS	V4	TILE_EN[8]	M19
GLOBAL_CLK	C19	TILE_EN[9]	M20
ISO_SIGNAL0	L18	TRIG[0]	G20
ISO_SIGNAL1	L19	TRIG[1]	R21
LED-2	R19	TRIG[2]	R20
LED-3	T19	TRIG[3]	T17
SFP_DISABLE	AA19	TRIG[4]	R18
SFP_FAULT	AA21	TRIG[5]	M22
SFP_LOS	AB21	TRIG[6]	M21
SFP_MOD	Y19	TRIG[7]	T18
SFP_TCLK_N	AB22	TRIG[8]	N19
SFP_TCLK_P	AA22	TRIG[9]	N20
SFP_TDATA_N	W21	TRIG_ISO	J18
SFP_TDATA_P	W20		

Netlist	Package Pin	Netlist	Package Pin
PISO[0]	C20	POSI[0]	D21
PISO[1]	D20	POSI[1]	E21
PISO[2]	G16	POSI[2]	B20
PISO[3]	G15	POSI[3]	B19
PISO[4]	V10	POSI[4]	U12
PISO[5]	V9	POSI[5]	U11
PISO[6]	Y9	POSI[6]	U10
PISO[7]	Y8	POSI[7]	U9
PISO[8]	V12	POSI[8]	AA7
PISO[9]	W12	POSI[9]	AA6
PISO[10]	W11	POSI[10]	Y6
PISO[11]	W10	POSI[11]	Y5
PISO[12]	B22	POSI[12]	C22
PISO[13]	B21	POSI[13]	D22
PISO[14]	A22	POSI[14]	G22
PISO[15]	A21	POSI[15]	H22
PISO[16]	AB9	POSI[16]	AB11
PISO[17]	AB10	POSI[17]	AA11
PISO[18]	AA8	POSI[18]	AB12
PISO[19]	AA9	POSI[19]	AA12
PISO[20]	C18	POSI[20]	B17
PISO[21]	C17	POSI[21]	B16
PISO[22]	F22	POSI[22]	E20
PISO[23]	F21	POSI[23]	E19
PISO[24]	B15	POSI[24]	A19
PISO[25]	C15	POSI[25]	A18
PISO[26]	D17	POSI[26]	A17
PISO[27]	D16	POSI[27]	A16
PISO[28]	AA4	POSI[28]	AB6
PISO[29]	Y4	POSI[29]	AB7
PISO[30]	AB4	POSI[30]	U4
PISO[31]	AB5	POSI[31]	T4
PISO[32]	F17	POSI[32]	E18
PISO[33]	G17	POSI[33]	F18
PISO[34]	E16	POSI[34]	D15
PISO[35]	F16	POSI[35]	E15
PISO[36]	V7	POSI[36]	Y11
PISO[37]	W7	POSI[37]	Y10
PISO[38]	W6	POSI[38]	V8
PISO[39]	W5	POSI[39]	W8

3 Programmable Logic Firmware

The PACMAN Zipline programmable logic (PL) firmware is implemented in a hierarchical fashion, with five unit-level modules that divide up the PACMAN features as follows:

- **Global**: this unit handles simple board-wide tasks: setting the power and tile enables, configuring the LEDs, reporting the top-level status, and firmware version.
- **CDR**: Clock and Data Recovery unit. This unit reconstructs the 62.5 MHz `cdr_clk` and timestamp.
- **TX**: this unit handles the configuration, operation, and monitoring of the 40 UART transmitters.
- **RX**: this unit handles the configuration, operation, and monitoring of the 40 UART receivers.
- **ATC**: this unit handles the ASIC timing and control signals (clock, reset, sync, and trigger) It produces a timestamp which is used by the RX unit to mark each data.
- **ADC**: this unit handles the onboard high-performance ADC available starting with PACMAN 1v5.

These five unit-level modules may appear to leave out some essential PACMAN features, notably control of voltage levels, power monitoring, and controlling the onboard multiplexer. These features are controlled over I2C by the processor (PS), and are therefore not part of the PL design.

The top-level design of the PL firmware is shown in Fig. 1. The Vivado corresponding block design (at interface level) as presented by the Vivado GUI[1], is shown in Fig. 3

This version of the documentation is synced with firmware version 3.6. The latest release of 3.X firmware is available on github at `https://github.com/mulhearn/pacman-firmware/tree/zipline-2023.2`

The modules are implemented as custom RTL modules written in VHDL. They contain submodules which are also written in VHDL. Every module adheres to project design rules[2], most importantly:

- Custom RTL modules do not use any vendor IP.
- Each module `xxx.vhd` has a testbench module `tb/xxx_tb.vhd` which demonstrates functionality to at least the level of initial hardware tests.
- Each module can be compiled and simulated using the lightweight open-source ghdl tool, independently of any vendor tools.

Both vendor IP and standard interfaces are featured prominently in the design:

- AXI-Lite (relatively slow 32-bit transfers) is used for custom registers.
- DMA (high performance direct memory access) is used for high-bandwidth TX and RX transfers.

[1]Diagram is an example block diagram only, and may not be from latest firmware version.

[2]The design rules worked well in practice, with the vast majority of bugs detected during GHDL simulation, relatively few remaining bugs detected on a Trenz evaluation module, and, nearly zero bugs (none that I can remember) detected using the actual hardware.

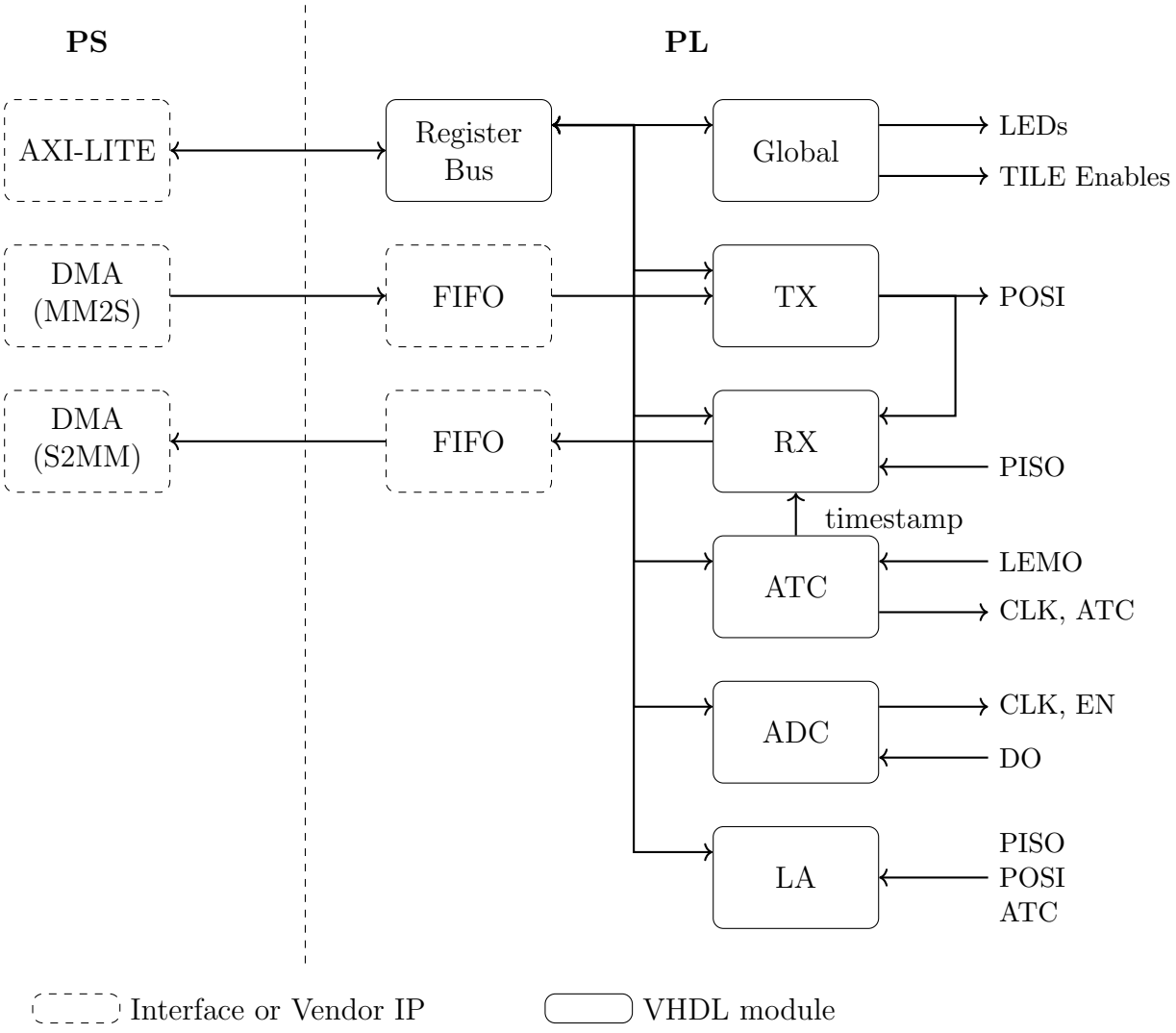


Figure 1: Top-Level Block Diagram of Zipline Firmware

- An AXI Stream FIFO is used to buffer the RX data between the RX unit and the DMA engine.
- BRAM (Block RAM) is used by the ADC unit to store the time series of conversion data.

The AXI interface is widely used throughout the design, and is well documented elsewhere. A critical point for understanding the AXI protocol is that data is transferred from a data producer to a data consumer when and only when a beat occurs. **A beat is defined as a clock-cycle during which the data producer asserts its valid signal and the data consumer asserts a busy signal.** There are further requirements, but this is sufficient to understand basic operation. For convenience, we use the same valid and ready handshake elsewhere in the design, even when we do not implement an AXI interface. The design uses one custom interface: the register bus (REGBUS). Each unit-level module is a secondary on the REGBUS, and is guaranteed to respond on the clock cycle immediately following a read or write request from the REGBUS primary. The PS drivers access the REGBUS interface via AXI-Lite.

3.1 Clocks

There are several logical clocks in the PACMAN firmware, which we define as:

- `axi_clk`: (100 MHz) AXI system clock, generated by PS, always available.
- `cdr_clk`: (62.5 MHz) generated by Clock and Data Recovery (CDR) subsystem from external fiber-optic input, which might not be present or valid.
- `sys_clk`: (TBD: 62.5-100 MHz) System clock, derived from either `cdr_clk` to `axi_clk`, with automatic fallback to the latter when the former is not present or invalid.
- `det_clk`: (TBD: 6.2-10 MHz), derived from `sys_clk` by a configurable divide factor of N (with nominally $N = 10$).

As shown in Fig.2 firmware components run in one of two clock domains: `axi_clk` and `sys_clk`. The PACMAN can be configured such that these are the same clock, but the firmware consistently treats them as independent, and always uses clock domain crossing for signals that pass between these two domains. A register read or write is initiated on the PS in the `axi_clk` domain. The register bus includes a CDC stage to reach all modules in the `sys_clk` domain. The TX and RX units are in the `sys_clk` domain, and reach the DMA engine, in the `axi_clk` domain, via FIFOs. Lastly, the global and CDR units are in the `axi_clk` domain.

If `sys_clk` is set to the 62.5MHz `cdr_clk` then the timestamps generated by the CDR are in the `sys_clk` domain. If we insist on deriving `sys_clk` from `cdr_clk` via a PLL to reach e.g. $100MHz$, then the timestamps will need to cross from the `det_clk` domain to the `sys_clk` domain. The more pessimistic scenario is shown in the figure. If we set `sys_clk` to `cdr_clk` then the PLL and timestamp CDC module are unnecessary. In this scenario, the only custom RTL needed for CDC is for the regbus.

The PACMAN communicates to the ASICs via RX and TX uarts running on `det_clk`. However, no PACMAN modules run on `det_clk` which from the PACMAN firmware perspective `det_clk` is a waveform in the `sys_clk` domain, with the falling edge at count int$(N/2)$ and the rising edge N. Since the PACMAN provides the UART clock `det_clk` to the UARTs, the phase relationships are fixed, and edge detection and metastability mitigation is unnecessary. Instead, each RX and TX channel registers at a fixed phase (from $0 \rightarrow N-1$) chosen to keep away from any metastability risk.

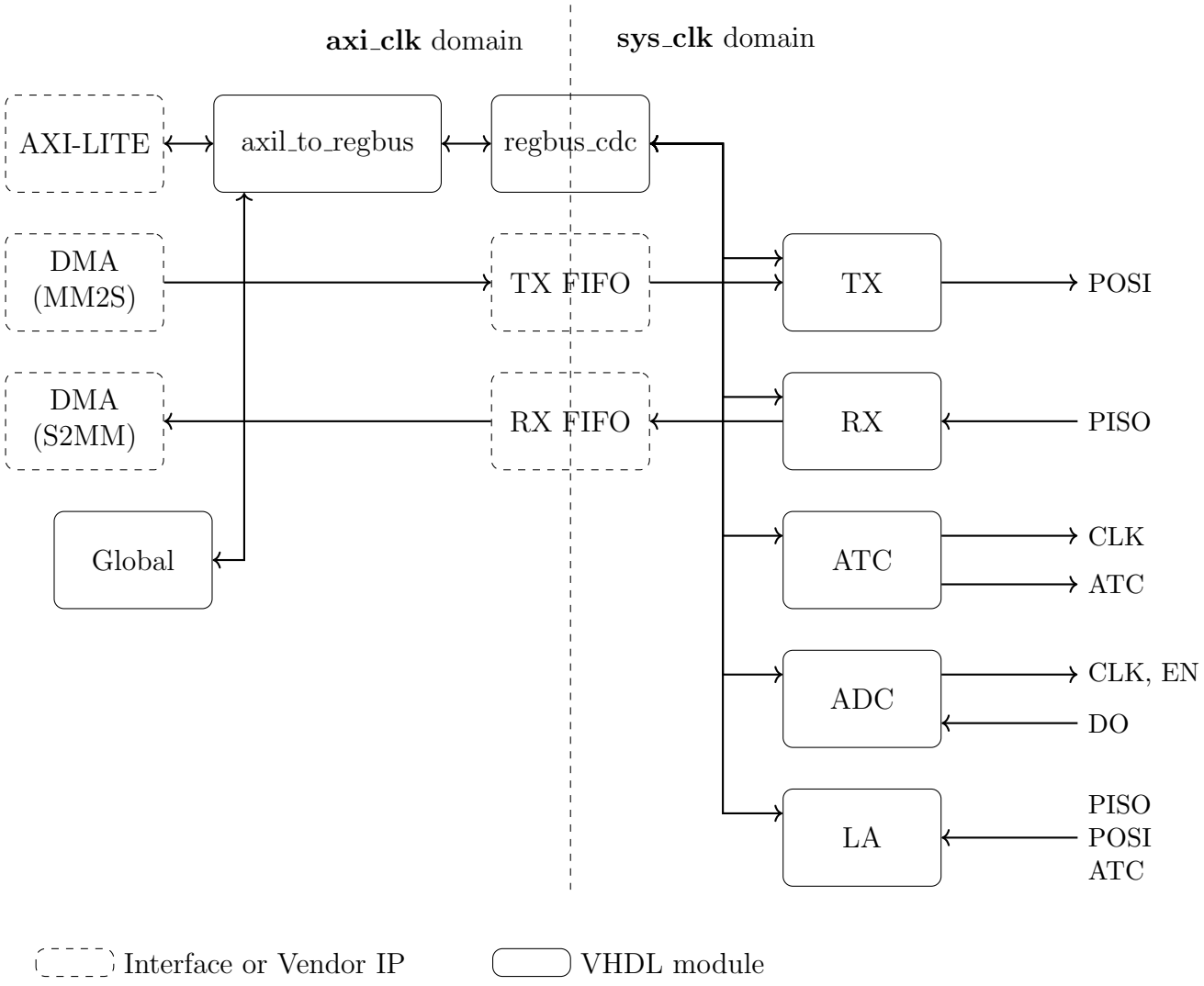


Figure 2: Clock Domains in the PACMAN firmware

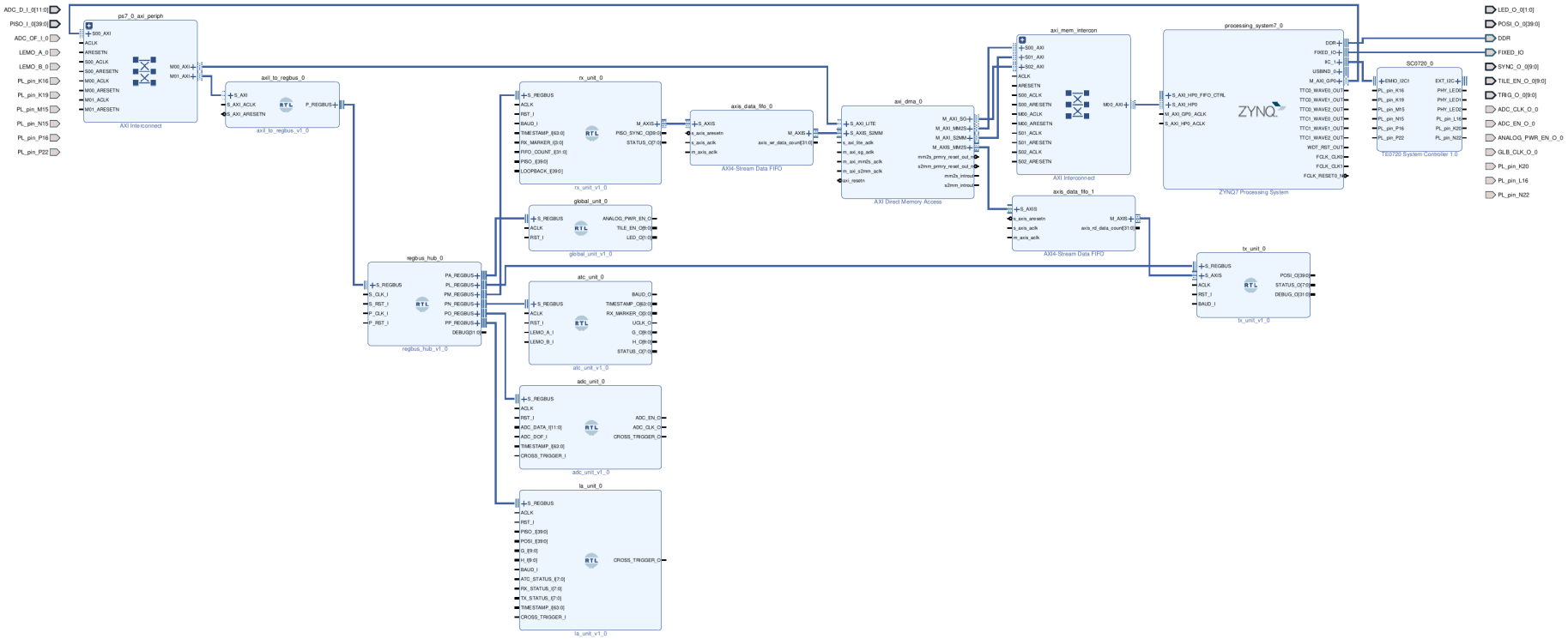


Figure 3: Vivado Block Design (Interfaces View)

Experience shows there is a wide plateau, and a global configuration suffices, but UART channel specific configuration is supported.

3.2 Registers

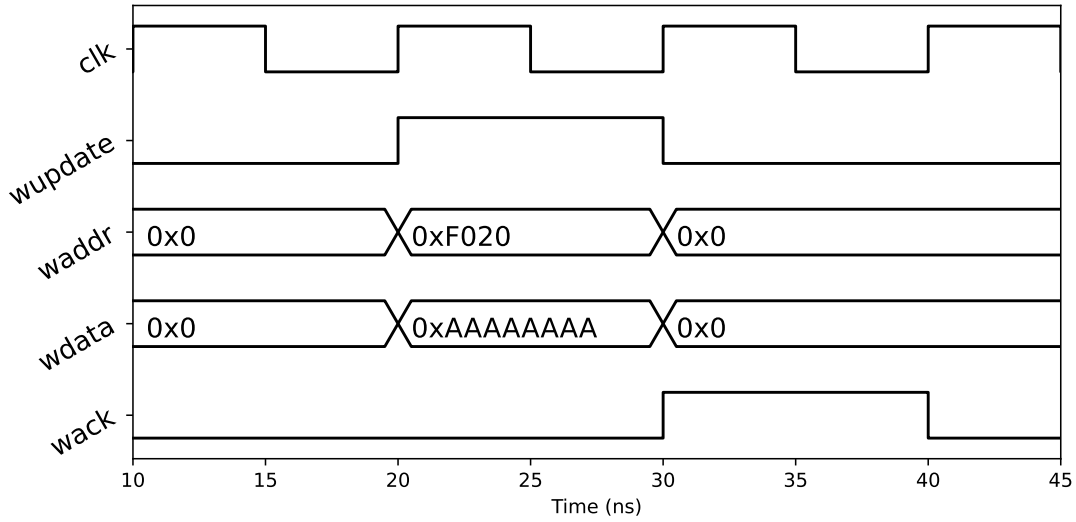


Figure 4: Single REGBUS write request (output of ghdl testbench)

Access to registers in the PL firmware is done via a simple custom register bus (REGBUS). A REGBUS write request is made by the single primary setting the 16-bit write address to that of the register of interest, the 32-bits of write data to the desired value, and the write update bit high. The secondary on the bus responsible for that register responds on the next clock cycle by setting the write acknowledge bit high and performing any other desired effect, such as setting a register to the value of the write data. A single write request is illustrated in Fig. 4. Command registers are implemented by setting a command signal high for one clock cycle after a write to the command register address, often ignoring the write data entirely.

A read request on the bus is made by the single primary setting the 16-bit read address to that of the register of interest and setting the read update bit high. The secondary on the bus responsible for that register responds on the next clock cycle by setting the 32-bit read data to the value associated with the register of interest, and setting the read acknowledge bit high. A single read request is illustrated in Fig. 5.

The register bus is implemented by a module (`axil_to_regbus.vhd`) which translates AXI-Lite requests from the PS to read and write requests on the REGBUS. Because FPGAs do not have internal tristate buffers, the bus implementation also requires a multiplexer `regbus_mux.vhd` to connect the REGBUS primary to multiple secondaries.

The 16-bit address space for PACMAN registers is allocated in a hierachical fashion, represented as

$$0\text{x}SRRR$$

with the most significant four bits 0xS defining the scope, and the remaining 12 bits 0xRRR defining the specific register within the scope. For example, the scratch A register (0x020), of the global unit (0xF) would be found at the 32-bit address:

$$0\text{xF020}.$$

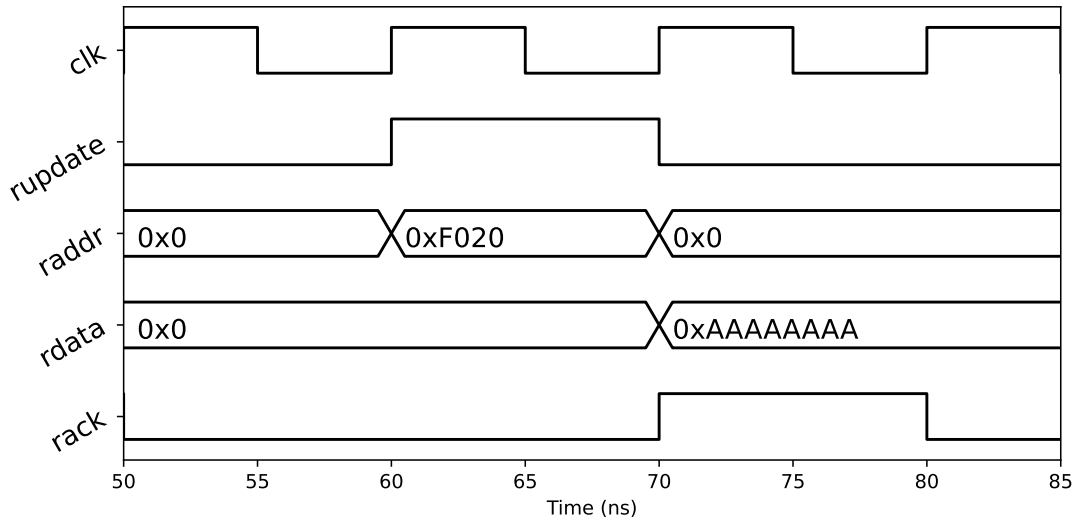


Figure 5: Single REGBUS read request (output of ghdl testbench)

Table 1: The most significant four bits of the register address, referred to as the scope, is allocated as shown.

Scope	Hexadecimal	Binary
Global	0xF	0b1111
ATC	0xE	0b1110
ADC	0xD	0b1101
TX	0x0-0x3	0b00XX
RX	0x4-0x7	0b01XX
LA	0xC	0b1100
(Reserved)	0x8-0xB	0b1000-0b1011

The scopes are allocated as shown in Table 1. The RX and TX unit are allocated a range of scopes to more readily accommodate the large number of registers associated with all 40 uarts (e.g. lost packet counts per uart channel). Further discussion of PACMAN registers is left to the subsections for each unit.

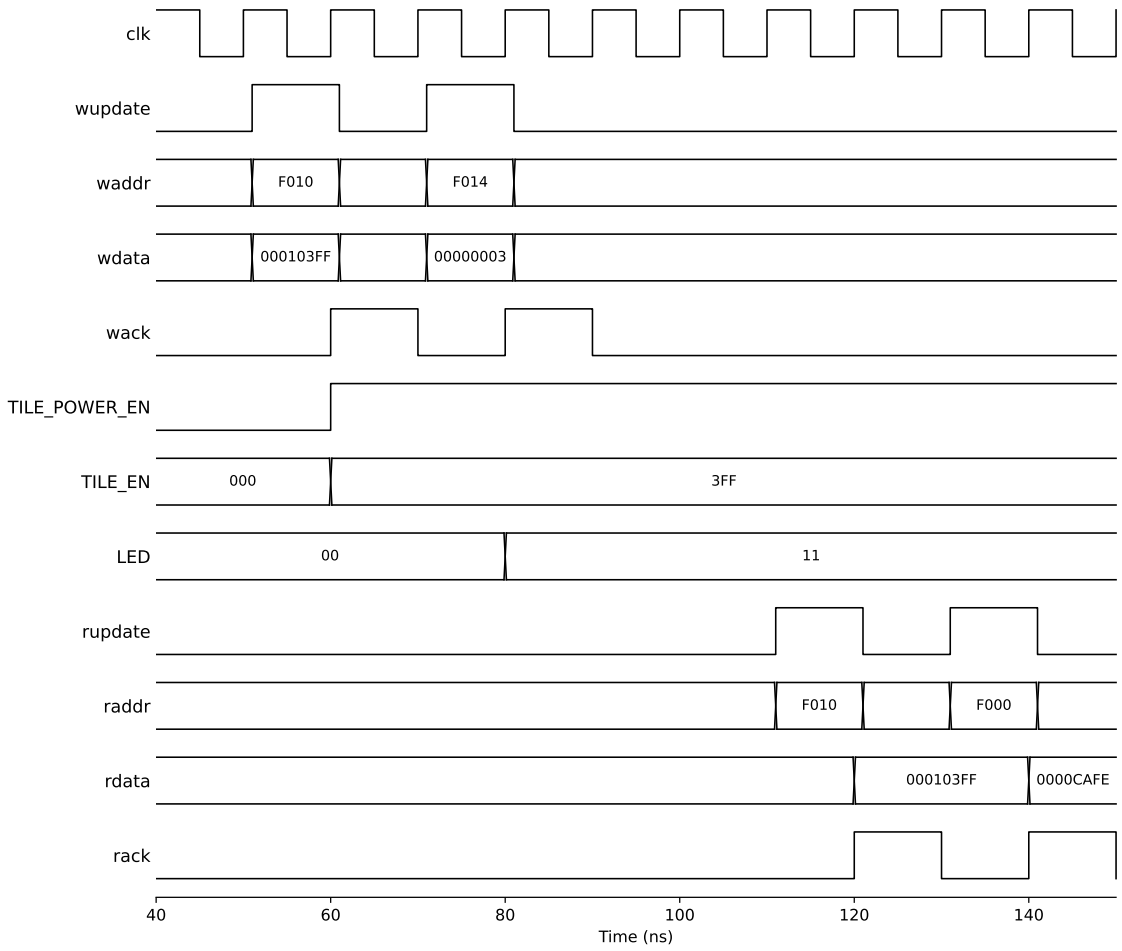
Table 2: Registers managed by the global unit

Register	Address	RO/RW
STATUS	0xF000	RO
ENABLES	0xF010	RW
LEDS	0xF014	RW
SCRATCH_A	0xF020	RW
SCRATCH_B	0xF024	RW
FIRMWARE_MAJOR	0xFF10	RO
FIRMWARE_MINOR	0xFF14	RO
FIRMWARE_PATCH	0xFF18	RO
HARDWARE_MAJOR	0xFF20	RO
HARDWARE_MINOR	0xFF24	RO
HARDWARE_PATCH	0xFF28	RO
SYNTHESIS_DATE	0xFF30	RO
GIT_HASH_UPPER	0xFF40	RO
GIT_HASH_LOWER	0xFF44	RO
BUILD_FLAGS	0xFF48	RO
VIVADO_MAJOR	0xFF50	RO
VIVADO_MINOR	0xFF54	RO

Table 3: Bit fields: description

Register	Field	Bits	Mask
ENABLES	TILE_ENABLES	0-9	0x000003FF
	TILE_POWER_ENABLE	16	0x00010000
LEDS	LED_3	0	0x00000001
	LED_4	1	0x00000002

3.3 Global Unit



The global unit handles simple board-wide tasks, and manages the registers listed in Table 2. The bit field interpretations for some registers are shown in Table 3. The read-only firmware registers can be used to determine the firmware version installed, and the read-only hardware registers can be used to check the hardware version for which the firmware was built. The FIRMWARE_BUILD and HARDWARE_BUILD registers are reserved. The read-write scratch registers SCRATCH_A and SCRATCH_B have no impact beyond storing and retrieving the register value, and are useful for testing the operation of the AXI-Lite and REGBUS interfaces.

Providing power and data I/O to the TILES requires enabling power to the tile power subsystem (called ANALOG_POWER_ENABLE in hardware schematics) as well as enabling the individual tile. These settings are available in the ENABLES register, using the bit fields TILE_POWER_ENABLE and TILE_ENABLES. While a tile is disabled, even with tile power enabled, the LVDS transmitters and receivers will be off, and both VDDD and VDDA are at zero. Setting the values of VDDD and VDDA, and monitoring currents and voltages on the board, is done over I2C, by the PS, and so is not part of the PL firmware design.

The PACMAN has two LEDs controlled by the PL. Their behavior is controlled by the LEDS register. In the current firmware, each LED is simply turned on or off directly, by the LED_3 and LED_4 bits, but in the future, they could be driven by specific conditions as well. Two additional LEDs are driven from the PS, and are not part of the PL firmware design. Two LEDs on the Trenz module are also controllable from the PS. The global unit also provides a global status register

(STATUS). In the current implementation, this register is implemented as a fixed pattern.

3.4 TX Unit

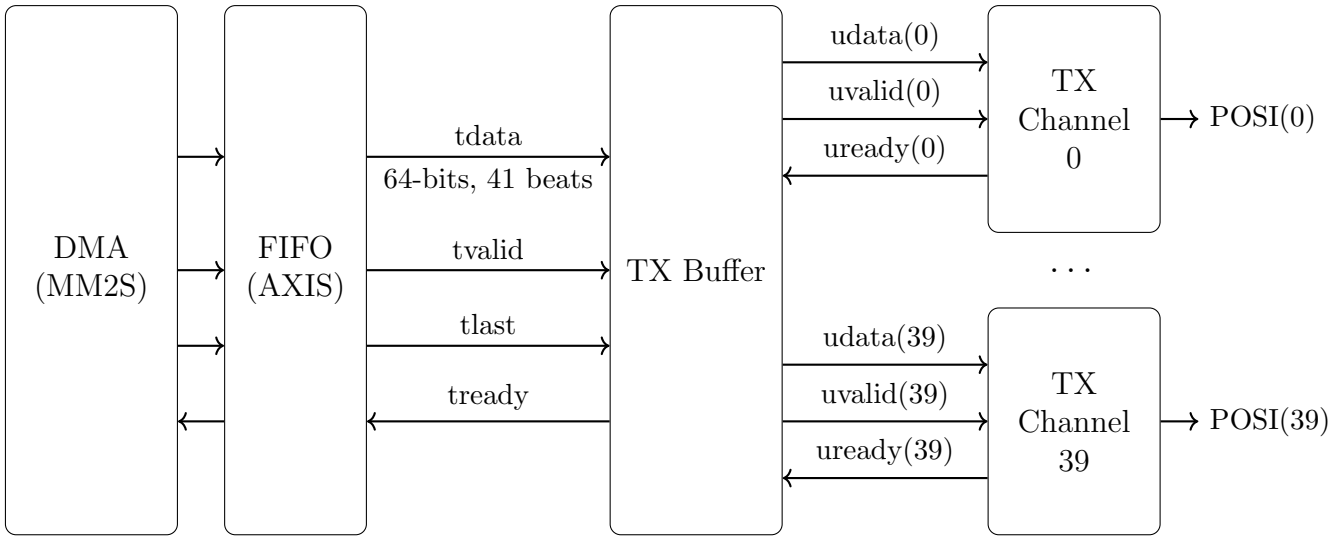


Figure 6: Data path in the TX Unit

The TX unit is responsible for configuration, operation, and monitoring of transmission to the tile cards via 40 UART channels. A block diagram of the TX unit is shown in Fig. 6. The data to be transmitted arrives as an AXI stream, which is filled by the PS using DMA.

Table 4: Contents of the TX DMA packet

Beat:	Contents:	Description
0	0x000000MMMMMMMMMM	M=UART Mask
1	0xDDDDDDDDDDDDDDDD	D=UART[0]
2	0xDDDDDDDDDDDDDDDD	D=UART[1]
	. . .	
40	0xDDDDDDDDDDDDDDDD	D=UART[39]

The TX Buffer sub-module contains a single buffer which holds incoming stream data until a complete DMA packet has been received. As shown in Table 4, a complete DMA packet consists of a 64-bit header and 40 64-bit words. The header contains a channel mask specifying which channels have data to transmit. The 40 64-bit words contain the 40 uart payloads. The DMA packet is the same size independent of how many UARTs are activated by the mask. Payloads from inactive UARTs are simply not used.

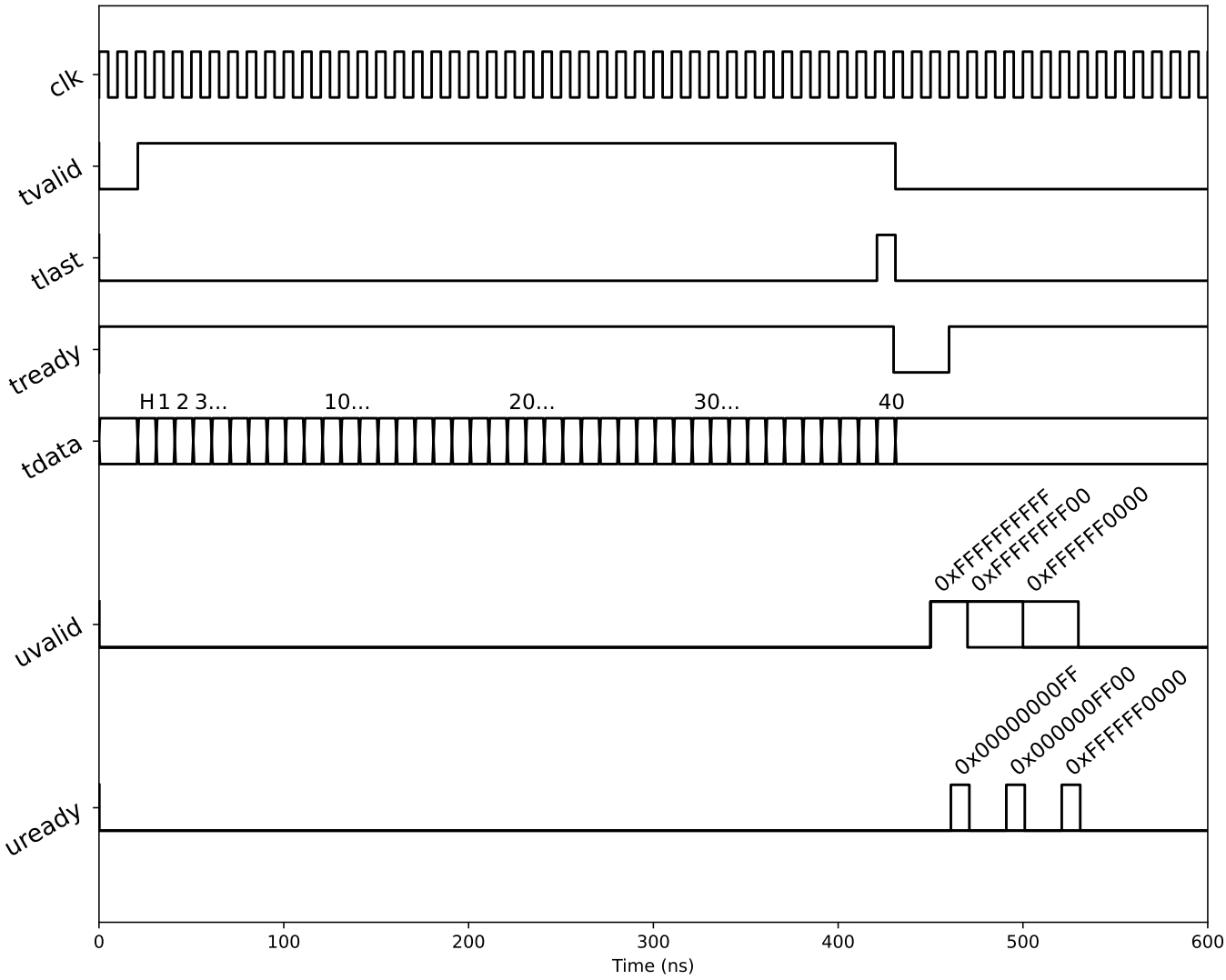
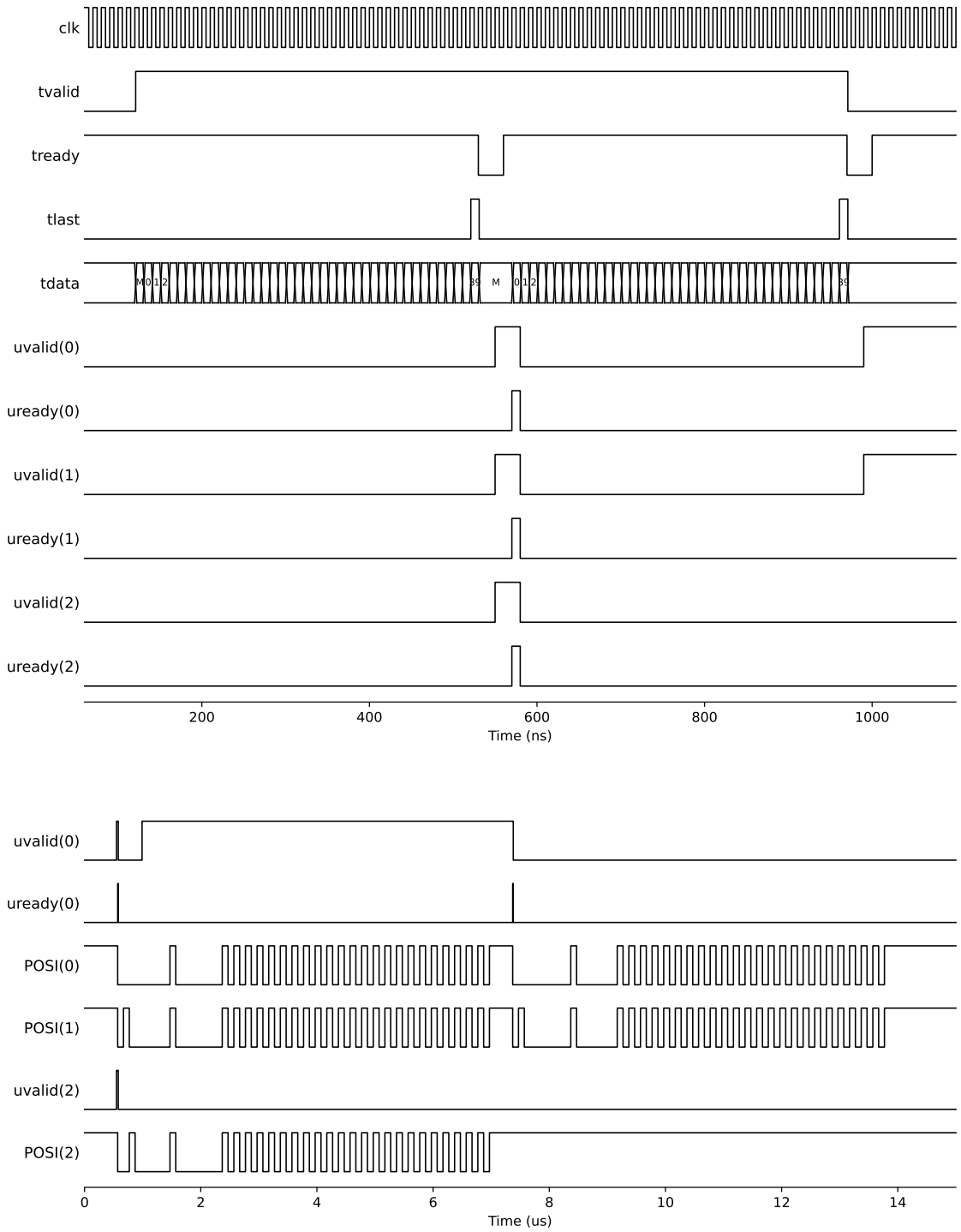


Figure 7: TX buffer handling a single DMA packet (output of ghdl testbench)



A timing diagram of the TX Buffer handling a single DMA packet is presented in Fig. 7. As discussed in the introduction, the AXI interface transfers data from producer to consumer when and only when both valid and ready are asserted on the same clock-cycle, called a beat. At the start, the TX buffer is empty, and so the stream ready (`tready`) signal is asserted by TX buffer. When data is available via DMA, the AXI stream asserts stream valid (`tvalid`). This continues for 41 beats. During the last beat, the stream asserts the last signal (`tlast`) indicating the end of the packet. The TX buffer will now contain a complete DMA packet, and so the TX buffer deasserts stream ready (`tready`) which halts the stream. It updates its own output buffer with the contents

of the DMA packet, and sets the valid bit for each UART according to mask received from the stream. On the next clock cycle, it asserts stream ready, so that the next DMA packet can begin loading in parallel with UART transmission.

The UART transmission is handled by 40 instances of the TX Channel sub-module, with one assigned to each UART channel. Even though this stage is not an AXI interface, the valid and ready handshake is used again, this time with one handshake per UART channel. When a TX channel is not currently transmitting, it loads the valid data presented by TX buffer into its internal shift registers for transmission, and asserts ready (`uready(i)`) on the next clock cycle.

Upon detecting a beat, the UART valid signal for the channel is cleared on the next clock cycle, indicating the data for that channel has been transferred to the shift register of the corresponding TX channel for transmission. When and only when all UART valid signals are cleared, the buffer is empty, the TX buffer asserts stream ready (`tready`) and, assuming data is available via DMA, the cycle begins again.

The slowest component in this chain is the UART transmission, which is bit serial. At 10 MHz, transmitting 64 bits, plus a start and stop bit, requires 660 100 MHz clock cycles. The DMA transfer of a single packet takes only 21 clock cycles[3] While each TX channel is transmitting, it deasserts its ready, which prevents a beat with TX Buffer, and causes TX Buffer to deassert stream ready once it has a DMA packet buffered. As soon as the current transmission is complete, the next transmission begins immediately. As long as new data is available via DMA, the transmitters run flat out. However, a configurable parameter inserts an artificial delay between each transmission cycle, which allows the TX speed to be throttled as needed to avoid overwhelming the ASICs.

You might notice in Fig. 7 that the timing in TX buffer is not particularly tight. The UART valid signals (`uvalid(i)`) and the Stream ready (`tready`) could in principle all go high one clock cycle earlier. The additional delay is due to pipelined logic. The AXI stream delivers the UART data with UARTs handled serially (two at a time). An initial stream reader stage parallelizes the stream data, and registers its output. Likewise, the calculation of the logical or across all of the UART valid signals (`uvalid(i)`) is registered as well. These calculations could be made combinatoric if tighter timing was needed. However, as discussed above, TX buffer is more than fast enough to keep up with the UART transmission at full speed, so we prefer the added robustness of the pipelined calculation.

The UART modules have been left unchanged from the legacy firmware. It was an intentional design choice to leave the PACMAN interfaces (to the ASICs, and to DAQ via pacman-server) unchanged during the initial development of the 3.X firmware, so that it could proceed without the need for extensive testing on ASICs and without any changes to the DAQ. The legacy UART design includes an unnecessary pause between start and stop bits, which increases the transmission time to 670 clock cycles (at 10 MHz). Unlike the TX buffer latency, this is at the timing bottle neck for the entire TX unit. However, this is still not problematic, because, as discussed above, we throttle down the TX transmission rate to avoid overwhelming the ASICs. An update to the UART firmware is planned for a future firmware release, but only after we have an extensive testbench available using actual ASICs.

Configuration and monitoring of the TX unit is performed over the AXI-LITE to REGBUS interface. Recall that the first four bits of the 32-bit address space is referred to as the scope, and is used to divide the 16-bit address space between different units. The TX unit is allocated the range of scopes from 0x0 to 0x3. Consistent with this assignment, the most significant bytes of a

[3]This is why we opt for a simple fixed length transmission, there is nothing further to be gained by e.g. zero suppression.

Table 5: All TX registers have leading two bits 0, and the next six bits define the channel. That least significant byte (XX below) specifies the register.

Channel	C	TX Register Address
UART 0-39	0x00-0x27	0x00XX-0x27XX
Broadcast	0x3B	0x3BXX
In Common	0x3F	0x3FXX

Table 6: Registers managed by the TX unit

Register	Address	RWC	Per UART
UART_STATUS	0x00	RO	Per UART
UART_CONFIG	0x04	RW	Per UART
UART_STARTS	0x20	RO	Per UART
UART_BEATS	0x24	RO	Per UART
LOOK_SELECT	0xA0	RW	In Common
LOOK_LOWER	0xA4	RO	In Common
LOOK_UPPER	0xA8	RO	In Common
BUFFER_STATUS	0xB0	RO	In Common
ZERO_CNTS	0xF8	CMD	In Common

TX register address are interpreted as:

$$0\text{b}00cccccc$$

where

$$C \equiv 0\text{b}cccccc$$

is the UART channel, taking values from 0 to 63. In addition to the 40 physical UART channels, assigned channels 0 to 39 (0x00 to 0x27), there are two notional channels: broadcast (0x3B) and in common (0x3F). This is summarized in Table 5. A single write to a UART register at the broadcast channel sets that registers for **all** UARTs to the specified value. Attempting to read from the broadcast channel is invalid. Registers associated to the entire TX unit are assigned to the notional channel "in common". Both read and write to the common channel, for defined common registers, is valid.

Table 7: Bit field descriptions for TX register

Register	Field	Bits	Mask
UART_STATUS	(Expert)	31-0	0xFFFFFFFF
UART_CONFIG	PHASE	7-0	0x000000FF
	(Reserved)	15-8	0x0000FF00
	DELAY	31-16	0xFFFF0000
BUFFER_STATUS	(Expert)	31-0	0xFFFFFFFF

The registers managed by the TX unit are presented in Table 6, and bit field interpretations for some of the registers are provided in Table 7. The UART configuration (UART_CONFIG) includes fields for a phase offset (in 10 ns ticks) and a clock divider (slow down factor) of the TX UART clock relative to the 100 ns clock sent directly to the ASICs. Normal operation is mode=1. For mode=0, the TX UART is off, and the channel holds ready high. The upper 16 bits of the register is the configurable delay (in 10 ns ticks) inserted between TX cycles. The UART status registers (UART_STATUS, one per UART) and TX buffer status (BUFFER_STATUS) interpretation is currently expert only, and not documented here, but the counts (explained below). More user friendly quantities (derived from the status registers) are available and described below.

A look register is provided to support reading the latest data received by each UART channel independently of DMA. To reduce FPGA usage, access is through a MUX that selects one UART channel at a time. The look registers (LOOK_LOWER and LOOK_UPPER) report the most recent 64-bit payload (lower and upper bits) of the UART channel selected via LOOK_SELECT. These buffers are used to verify the operation of the DMA to UART channel transmission, and do not test the actual operation of the UART transmitter. Operation of the UART itself can be verified by software loopback of the UART output to the RX UARTs (as described in the RX section) or by physical loopback. The number of beats encountered (UART_BEATS) and the number of transmission start bits sent (UART_STARTS) is available for each UART. These counts are cleared upon reset and by writing the zero counts (ZERO_CNTS) register.

The TX firmware has been demonstrated in the hardware (using both bare-metal applications and Linux) to run continuously at maximum speed without any bit errors.

3.5 RX Unit

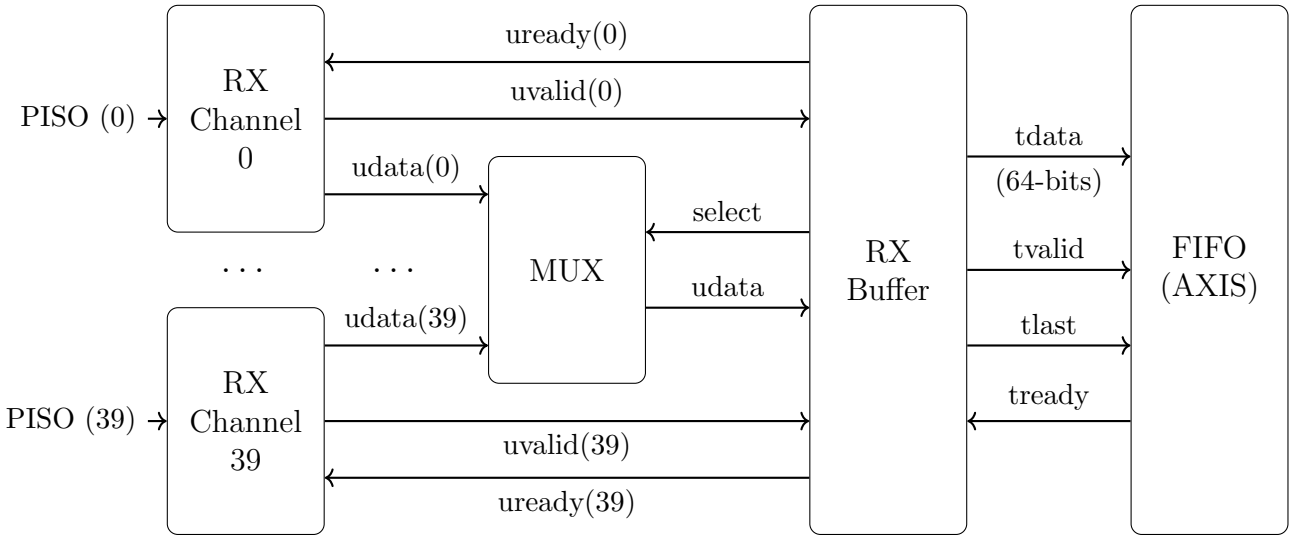


Figure 8: Data path in the RX Unit. Timestamps from each channels are MUXed in a similar fashion for inclusion in the stream. A secondary output of the MUX is used to provide a look register.

The RX unit receives UART packets from the tile cards and forwards the data to memory in the processor (PS). A block diagram of the RX unit is shown in Fig. 8. Incoming UART packets, on PISO(0) to PISO(39), are received by 40 independently operating RX channels. The RX buffer combines the output of all 40 UART channels and writes to an AXI stream FIFO. The DMA engine reads from the FIFO and places the data in memory accessible from the processor (PS).

Each UART channel has a single 64-bit buffer which is marked valid upon a complete transfer from the UART receiver. In addition to the 40 UART channels, additional RX channels are assigned for trigger[4] and sync words. The RX unit uses a valid and ready handshake, as in the AXI protocol, which was described in previous sections. Valid data is streamed using turn-based round-robin scheduling, according to the schedule according to the turn assignments shown in Table 8. The stream width is 64 bits, but the word size is 192 bits, so each turn lasts for three 64-bit fragments. Neither the turn count, nor the fragment count , advance when the stream is busy.

The RX buffer is a FSM with the following states:

- **IDLE:** waiting for arrival of valid data.
  ⇒ On arrival of valid data, move to **STREAM** state.
- **WAIT_STATE:** all valid data streamed, wait for new data or timeout ⇒ On arrival of valid data, move to **STREAM** state
  ⇒ If configurable timeout occurs, move to **TRAILER** state
- **STREAM:** turn-based round-robin streaming of data to FIFO. Each RX channel is assigned one turn which lasts long enough to stream three 64-bit fragments when valid data is available and the stream is not busy.
  ⇒ If the number of words sent exceeds a threshold, move to **TRAILER** state.
  ⇒ If no more valid data is available, move to **WAIT_STATE**

[4]trigger words not implemented in current firmware

Table 8: Turn counter assignments for the RX buffer and default word type character.

Counter c:	Stream word for:	Word type (ASCII)
0x00-0x27 (0-39)	udata(c)	0x44 (D)
0x28 (40)	sync heartbeat	0x53 (S)
0x29 (41)	sync rollover	0x53 (S)
0x2A (42)	marker 2	0x54 (T)
0x2B (43)	marker 3	0x00
(after 43)	trailer	0x4C (L)

- **TRAILER:** stream trailer word. At the end of the packet, send three 64-bit fragments to write a 192-bit trailer reporting the number of 192-bit words streamed (not including the TRAILER), and set LAST bit on the final fragment of the trailer.
  ⇒ Return to **IDLE** state after TRAILER is streamed.

Data for the channel corresponding to the current turn is selected via the MUX. After valid data for a channel has been buffered for streaming, the ready bit is set for that channel. On receiving ready, each UART channel clears its valid bit. If new data arrives on the RX channel before the valid bit is cleared (via ready) the packet is lost, which is noted by the lost bit in the UART status. Counters track the number of lost packets for each UART (which should be zero during normal operation).

Although the data is streamed one word at a time, many words are assembled into a single DMA packet (as marked via the LAST bit). The size of the DMA packet is configurable based on a maximum number of words and a timeout. In both cases, a zero is no condition (no timeout / no maximum). The timeout is not checked while in stream state, so e.g. a timeout of one (default setting) will send data from all channels with valid data once valid data from at least one channel is available.

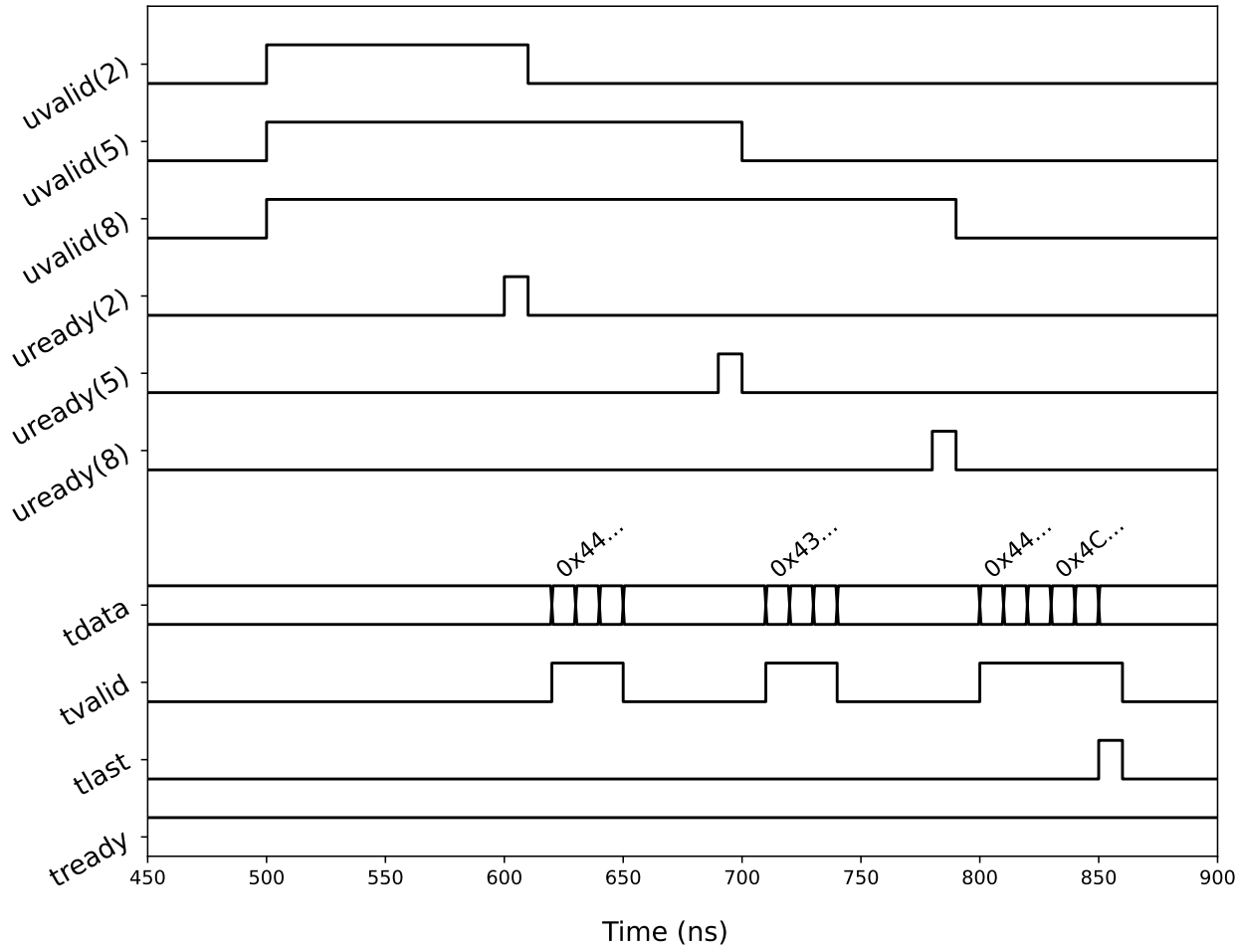
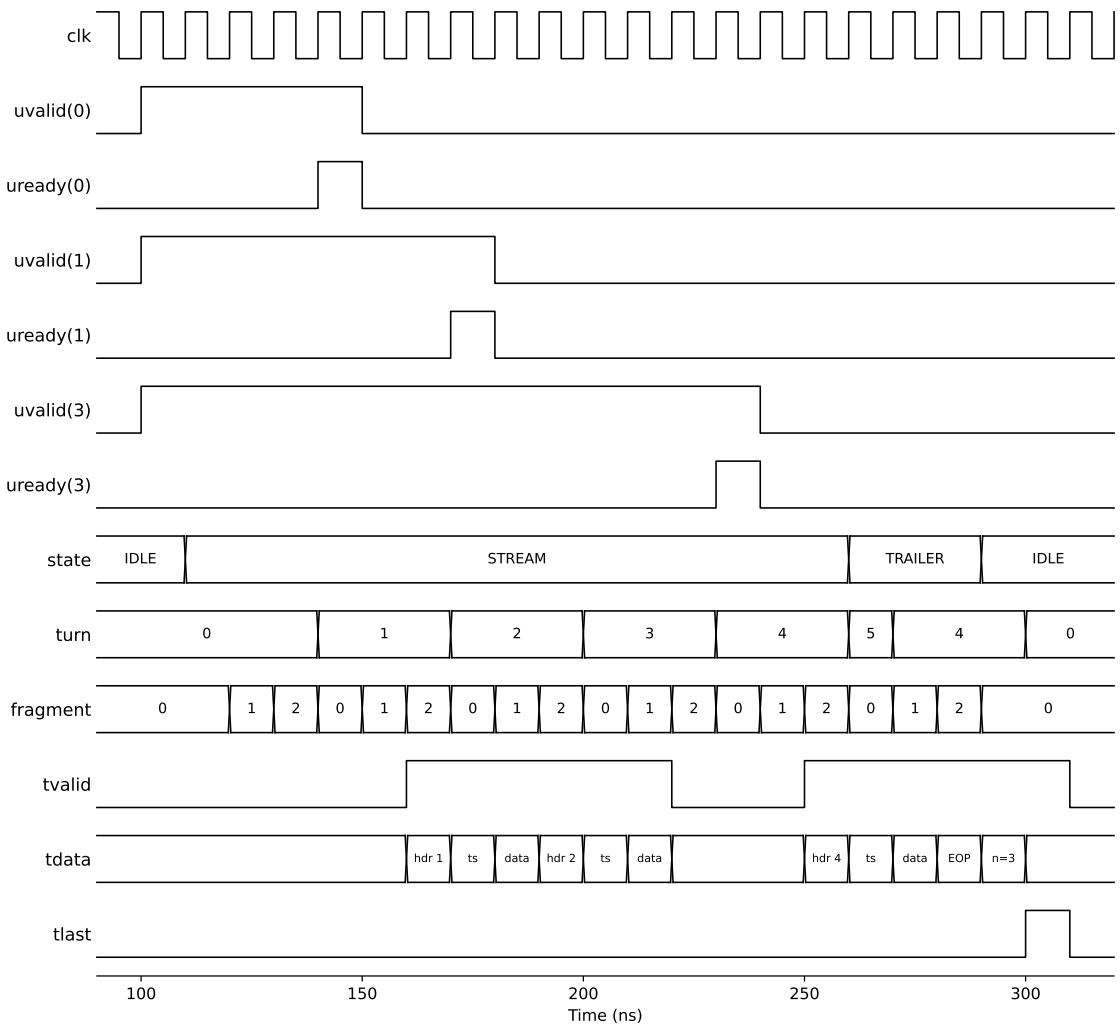


Figure 9: RX buffer handling a single DMA packet containing data received from three UART channels (output of ghdl testbench) For simplicity, only a single indicative byte of the stream data (tdata) is shown (e.g. 0xAA).



A timing diagram from a GHDL simulation of the RX buffer operation is presented in Fig. 9. At the start, only UART channels 2,5, and 8 have data available and are each therefore asserting their valid signal. On the turn assigned to each UART channel, the RX buffer asserts ready for that channel. On the next clock cycle, the RX channel deasserts its valid. Then, on the next clock cycle, the RX buffer presents the data to the stream. The stream writer asserts stream valid (`tvalid`) for three clock cycles while these three words are added to the stream. During turns not assigned to channels with valid data, the stream valid is deasserted. In the example shown, the RX buffer has been configured with the maximum number of words per packet set to three. Therefore, the trailer is sent immediately after the third word.

In the latest firmware, the word type for UART data is determined dynamically based on the actual content of the ASIC data payload. This is intended to mark configuration replies with the character C (0x43) and data with the character D (0x44). By default, all uart data is marked D, but in this example, it has been configured to separate configuration replies from data. You can see this is the timing diagram as 0x44,0x43, and 0x44. The trailer word header (!) is also configurable, and here it has been marked with an ASCII L (0x4C).

In the demonstration, the FIFO is never full, and so the stream always asserts stream ready (`tready`). If the DMA engine were to fall behind, then the FIFO could fill, and the stream would deassert stream ready. This would cause the RX buffer to refrain from asserting any UART ready (`uready(i)`) signals, which prevents any UART valid (`uvalid(i)`) from being cleared. If new data

Table 9: All RX registers have leading bit 0 and next leading bit 1, and the next six bits define the channel. That least significant bytes (XX below) further specifies the register.

Channel	C	RX Register Address
UART 0-39	0x00-0x27	0x40XX-0x67XX
Broadcast	0x3B	0x7BXX
In Common	0x3F	0x7FXX

arrives on any RX channel before its valid bit is cleared (via ready) the packet is lost, which is noted by the lost bit in the UART status. Counters track the number of lost packets for each UART (which should be always zero during normal operation).

Configuration and monitoring of the RX unit is performed over the AXI-LITE to REGBUS interface. In a similar fashion as for the TX unit, the RX unit is assigned a range of scopes (from 0x4 to 0x7) so that the most significant byte of the address can be interpreted as:

$$0\text{b}01cccccc$$

where

$$C \equiv 0\text{b}cccccc$$

is the UART channel, taking values from 0 to 63. As for the TX unit, in addition to the 40 physical UART channels, assigned channels 0 to 39 (0x00 to 0x27), there are two notional channels: broadcast (0x3B) and in common (0x3F). This is summarized in Table 9.

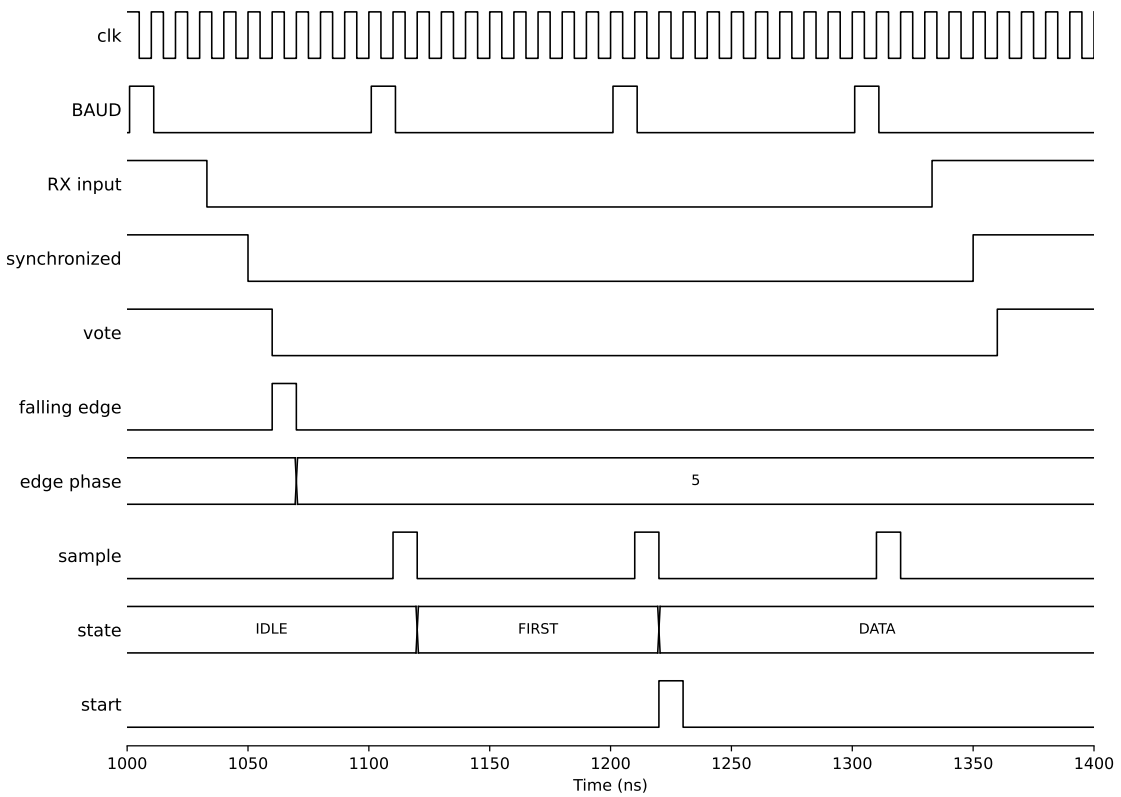
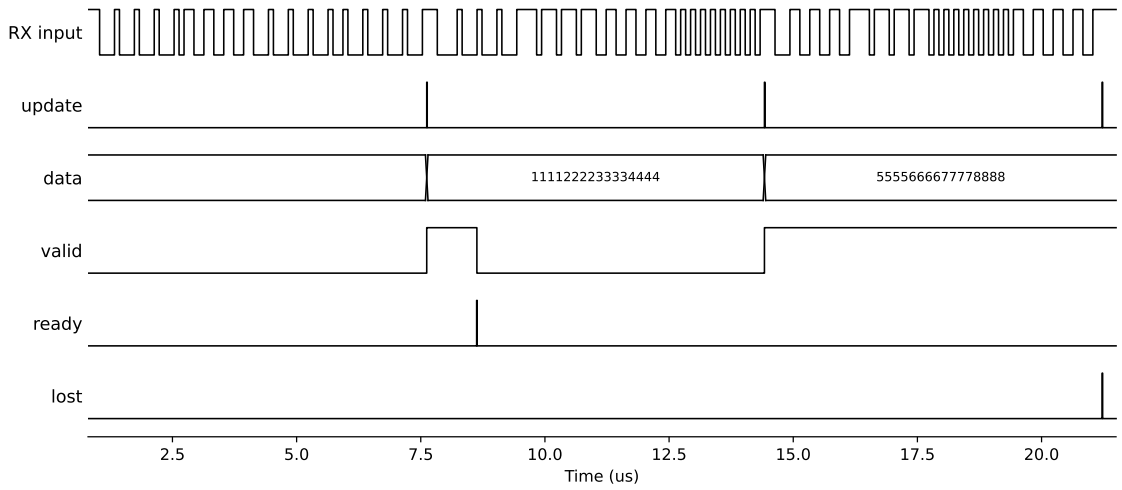


Table 10: Registers managed by the RX unit

Register	Address	RWC	Per UART
UART_STATUS	0x00	RO	Per UART
UART_CONFIG	0x04	RW	Per UART
UART_CHAN	0x08	RW	Per UART
UART_STARTS	0x20	RO	Per UART
UART_BEATS	0x24	RO	Per UART
UART_UPDATES	0x28	RO	Per UART
UART_LOST	0x2C	RO	Per UART
UART_FRAME_ERR	0x30	RO	Per UART
UART_NOISE	0x34	RO	Per UART
PATTERN_LOWER	0x80	RW	In Common
PATTERN_UPPER	0x84	RW	In Common
PATTERN_DELAY	0x88	RW	In Common
PATTERN_CONFIG	0x8C	RW	In Common
PATTERN_STATUS	0x90	RO	In Common
PATTERN_STARTS	0x94	RO	In Common
PATTERN_STOPS	0x98	RO	In Common
LOOK_SELECT	0xA0	RW	In Common
LOOK_LOWER	0xA4	RO	In Common
LOOK_UPPER	0xA8	RO	In Common
BUFFER_STATUS	0xB0	RO	In Common
BUFFER_CONFIG	0xB4	RW	In Common
BUFFER_ENABLES	0xB8	RW	In Common
PACMAN	0xBC	RW	In Common
HEARTBEAT_CONFIG	0xC0	RW	In Common
ROLLOVER_CONFIG	0xC4	RW	In Common
WORD_TYPE_LUT	0xC8	RW	In Common
HEADER_A	0xD0	RW	In Common
HEADER_B	0xD4	RW	In Common
HEADER_C	0xD8	RW	In Common
HEADER_D	0xDC	RW	In Common
EOP_HEADER	0xE0	RW	In Common
FIFO_CNT	0xF0	RO	In Common
FIFO_MAX	0xF4	RO	In Common
ZERO_CNTS	0xF8	CMD	In Common

Table 11: Bit field descriptions for RX register

Register	Field	Bits	Mask
UART_STATUS	(Expert)	31-0	0xFFFFFFFF
UART_CONFIG	PHASE	7-0	0x000000FF
	INPUT	11-8	0x00000F00
	MODE	13-12	0x00003000
	FLAGS	15-14	0x0000C000
	PERIOD	23-16	0x00FF0000
	(Reserved)	31-24	0xFF000000
BUFFER_STATUS	(Expert)	31-0	0xFFFFFFFF
BUFFER_CONFIG	TIMEOUT	15-0	0x0000FFFF
	MAX_WORDS	31-16	0xFFFF0000
BUFFER_ENABLES	HEARTBEAT	0	0x00000001
	ROLLOVER	1	0x00000002
	MARKER_2	2	0x00000004
	MARKER_3	3	0x00000008



The registers managed by the RX unit are presented in Table 10, and bit field interpretations for some of the registers are provided in Table 11. The UART configuration (UART_CONFIG) includes fields for a phase offset (in 10 ns ticks) and a clock divider (slow down factor) of the RX UART clock relative to the 100 ns clock sent directly to the ASICs. Normal operation is MODE=1. For MODE=0, the RX unit is squelched: updates from the UART receivers are ignored, the channel's output is set to zero, and its valid signal is deasserted. The RX input is configured by the INPUT parameter. Normal operation is INPUT=0 which selects the UART inputs (PISO). Setting INPUT=1 selects the output of the TX unit for the corresponding channel, providing a software loopback option. Setting INPUT=2 fixes the input to 0, and INPUT=3 fixes the input to 1.

The RX buffer configuration register (BUFFER_CONFIG) includes the TIMEOUT and MAX_WORDS fields, which control the end of packet determination as described above. The sync word behavior is configured by the number of clock cycles between heartbeats (HEARTBEAT_CONFIG) and the

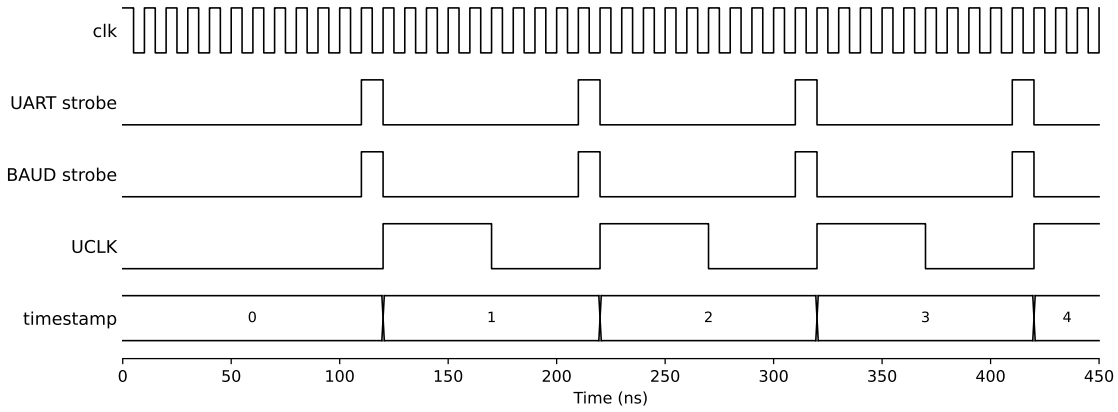
timestamp at which a rollover is recorded (ROLLOVER_CONFIG). The heartbeat and rollover SYNC messages are only included in the DMA stream if the corresponding enables are set in BUFFER_ENABLES. The UART status registers (USTATUS, one per UART) and RX buffer status (BSTATUS) interpretation is currently expert only, and not documented here.

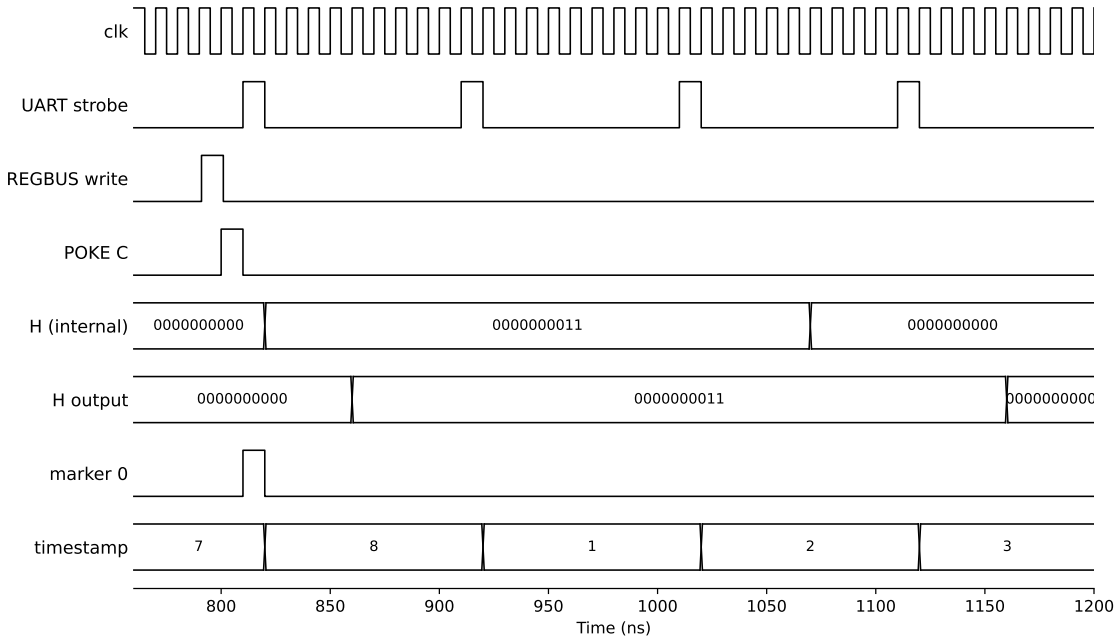
A look register is provided to support reading the latest data received by each UART channel independently of DMA. To reduce FPGA usage, access is via a second output channel of the same MUX used by the RX buffer. The look registers (LOOK_LOWER and LOOK_UPPER) report the most recent 64-bit payload (lower and upper bits) of the UART channel selected via LOOK_SELECT.

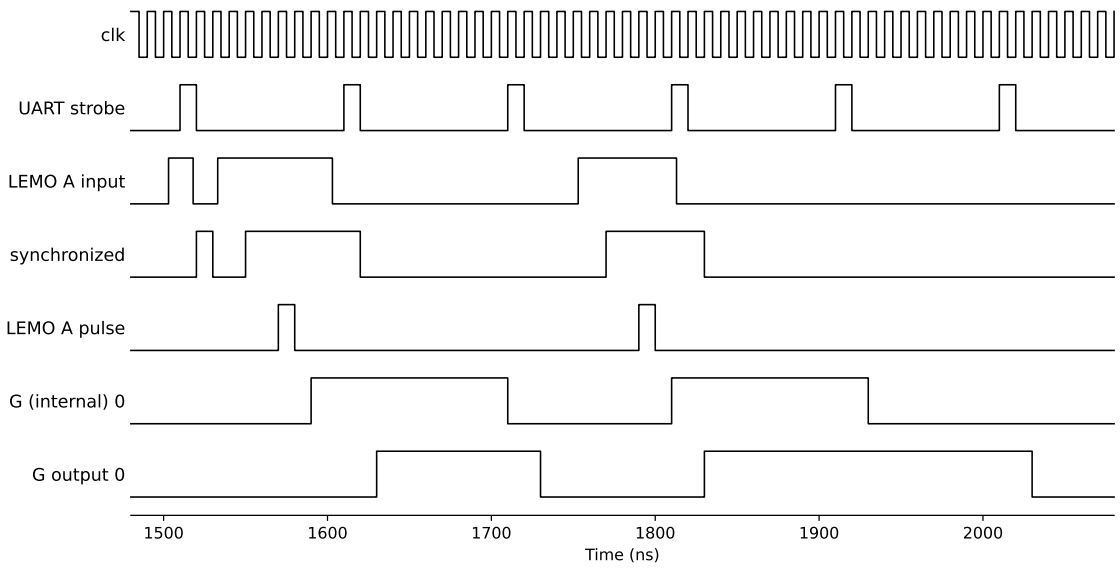
Several counts are maintained for each UART: the number of start bits received (UART_STARTS), beats observed (UART_BEATS), and successful receptions (UART_UPDATES). The number of lost packets (UART_LOST), as indicated by the lost bit being asserted, are likewise counted. The counts are cleared upon reset and by writing the zero counts (ZERO_CNTS) register. The current number of words (FIFO_CNT) in the RX FIFO is available, as is the maximum number of words (FIFO_MAX) since the last reset of zero counts command.

3.6 Timing Unit

The timing unit is responsible for the 10 MHz clock (UCLK) sent to the ASICs and producing a timestamp, synchronous with UCLK, which is added to the incoming data. The timing unit also handles timing control signals sent to the ASICs, such as SYNC, RESET, and TRIG signals. A fully featured Timing Unit is available in the latest firmware, but we expect at least one round of substantial revisions before this design stabilizes, and so we will not document this unit further for this version.







3.7 ADC Unit

The ADC unit is responsible for operating the on-board high-performance ADC. A fully featured Timing Unit is available in the latest firmware, but we expect at least one round of substantial revisions before this design stabilizes, and so we will not document this unit further for this version.

4 PACMAN Server

4.1 DAQ Interface

The PACMAN Server interface to the DAQ is a virtual address spaces. Virtual addresses refer to specific features, for example, powering VDDD of tile 1 to a specific voltage level. The use of a virtual addresses allows for the most consistent support across hardware versions. Direct (non-virtual) interface to the AXI LITE address space is provided at upper region of the address space, as well as direct access to the I2C bus.

Word Type	Char	Field	Symbol	Bytes	Bits	Notes
PING	'P'	PACMAN ID	P	1	8	Optional for request
READ	'R'	PACMAN ID	P	1	8	Optional for request
		Address	A	4	32	
		Read Value	R	4	32	0 for request
WRITE	'W'	PACMAN ID	P	1	8	Optional for request
		Address	A	4	32	
		Write Value	W	4	32	
DATA	'D'	PACMAN ID	P	1	8	Optional for TX
		Channel	C	2	16	
		Timestamp	T	8	64	Optional for TX
		Data Payload	D	8	64	
CFG	'C'	PACMAN ID	P	1	8	Optional for TX
		Channel	C	2	16	
		Timestamp	T	8	64	Optional for TX
		Data Payload	D	8	64	
SYNC	'S'	PACMAN ID	P	1	8	
		Sync Type	S	1	8	
		Clock Source	L	1	8	
		Timestamp	T	8	64	
		Status	U	4	32	
TRIG	'T'	PACMAN ID	P	1	8	
		Trigger Type	G	1	8	
		Trigger Source	I	1	8	
		Timestamp	T	8	64	
ERR	'E'	PACMAN ID	P	1	8	
		Timestamp	T	8	64	
		Error Code	O	4	32	

Table 12: Constituent fields of PACMAN words by word type (with associated character). Each word is 24 bytes (192 bits) but padding is not listed here. The size in bytes (and bits) and the single letter symbol used for each active field is listed.

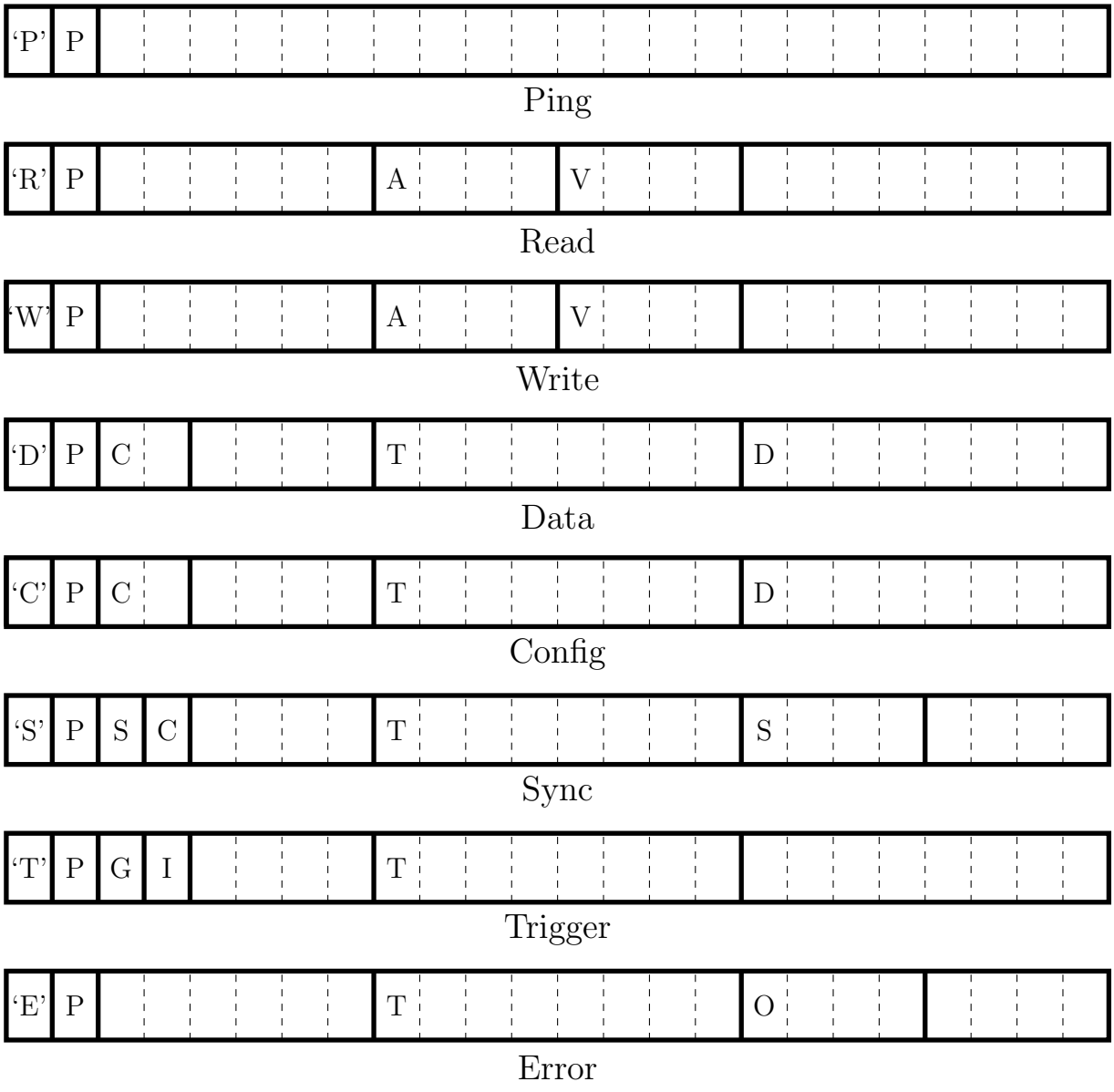


Figure 10: Layout of PACMAN message words by word type, showing field locations and sizes, labeled by the single character symbol from Table 12. The space between each pair of lines represents one byte, and each word is 24 bytes wide (192-bits). The least significant bit is to the left, the least significant byte is a character field used to indicate the word type.

Command	Description
ping	Reply pong
report_version_details	Report firmware version and related info
read_pacman_id	Reply with current PACMAN ID
set_pacman_id id=<id>	Set PACMAN ID to <id>
set_vddd tile=<tile> mv=<mv>	Set tile <tile> VDDD to <mv> mV
set_vdda tile=<tile> mv=<mv>	Set tile <tile> VDDA to <mv> mV
enable_tile_power	Enable main analog power rails for all tiles
disable_tile_power	Disable main analog power rails for all tiles
enable_tile tile=<tile>	Enable local power and logic for tile <tile>
disable_tile tile=<tile>	Disable local power and logic for tile <tile>
read_vddd tile=<tile>	Reply with measured VDDD for tile <tile>
read_vdda tile=<tile>	Reply with measured VDDA for tile <tile>
read_enables	Reply with global and per-tile enable status
send_full_reset mask=<mask>	Send full reset to tiles enabled in <mask>
send_internal_reset mask=<mask>	Send internal reset to tiles enabled in <mask>
send_sync_timestamp mask=<mask>	Send sync timestamp to tiles enabled in <mask>

Table 13: High-level DAQ interface commands for the PACMAN card. All commands use key=value syntax. Parameters shown in angle brackets (e.g. <tile>) indicate user-specified values. Tile enable operates in two stages: enable_tile_power applies global power rails, and enable_tile activates individual tiles.

5 Developer Workflows

6 Quality Assurance and Quality Control

The PACMAN card quality assurance and quality control (QA/QC) regimen relies on several complementary techniques:

- **Loopback:** Loopback is generally the most effective method of feature verification. The loopback mindset first drives the design toward adequate feedback and test input mechanisms, and then tests both the feature and its corresponding feedback or test input mechanism simultaneously. Loopback paths in hardware test components and solder quality through the entire path.
- **Fault Localization:** Designing for fault localization forces a modular design that lends itself to effective QC. For an important example, loopback can be performed in the hardware, or in firmware (just prior to leaving/ entering the device), or in the CPU, so that the source of a failure can immediately be localized to hardware, programmable logic, or software.
- **Design Review:** The PACMAN design team includes a lead engineer, and a consulting engineer, and a principal investigator. All three understand the board down to the last net list. The PACMAN design is also periodically subjected to independent review.
- **Prototype Systems:** The most effective form of quality control is operation in a working system, and the PACMAN card has been used continuously in prototype systems since the first version in 2020.

6.1 Feature Verification Overview

Here we list design features of the PACMAN card that are tested during hardware verification, in most cases by a loopback mechanism. Specific details on performing the relevant tests are discussed in the hardware checkout section.

- The LEDs are tested with blink patterns.
- The provision of tile voltages VDDD and VDDA is verified by onboard current and voltage monitoring ADCs.
- The POSI and PISO (Digital I/O to Tiles) signals are tested via loopback using both test patterns and pseudorandom numbers.
- The onboard high-performance ADC is tested by connecting both the positive and negative terminals to independently controlled DAC outputs.
- The CLK, RESETN/SYNC/TRIG signals (Digital I/O to Tiles) are tested by loopback to the onboard ADC (DONE) through the ASIC Monitoring Channels. In addition to these signals, this therefore tests MUX and the Analog Monitoring Signals.
- The CLK, RESETN/SYNC/TRIG signals (Digital I/O to Tiles) are tested by loopback to PISO channels (Not yet implemented, test above suffices).
- The Front-Panel SMA connectors and MUX are checked by leaving them open or connecting a resistor in software and confirming the resistance with a DMM (Not yet in Linux)

- The Front-Panel LEMO connectors are checked by driving them with external TTL signals and checking counters in the software.

The PACMAN card hosts a linux system, so a broad spectrum of test software is immediately available to us. Here we list what we consider to the be the most important tests of the Linux host. Specific details on performing the relevant tests are discussed in the hardware checkout section.

- Successful system BOOT, visible via UART.
- Successful remote login via ethernet.
- Successful transfer of pseudo-random data via ethernet (scp) and the UART terminal (lrzsz).
- Repeated read and write of pseudo-random numbers to the SD card.

Some of the tests described in this section require configuring the PACMAN card in ways that are not appropriate for a card installed in the production system. For this reason, the PACMAN card includes an expert mode enable two pin jumper. When a PACMAN card is installed in the production system, this jumper is removed, and the software refuses to provide inappropriate test configurations.

6.2 Requirement Verification

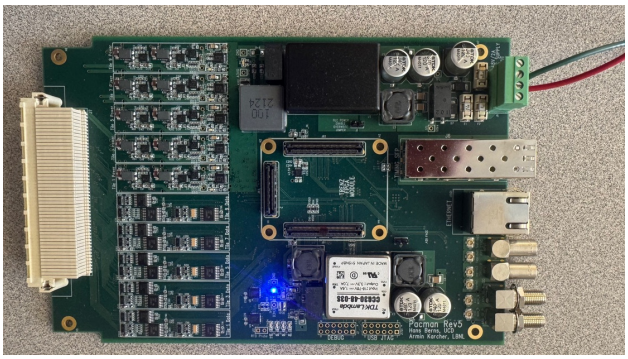
In addition to feature operation, the PACMAN has several performance requirements which require quantitative verifications:

- The provision of voltage and power across the required ranges is tested by applying the appropriate load resistance to VDDD and VDDA.
- The common mode rejection feature of the DC voltage supply is tested by injecting larger than expected common mode noise at the input and measuring the provided DC voltage (This test will be performed on a subset of the PACMAN cards, to validate the design.)
- The bit error rate is measured during loopback testing of the digital I/O systems.
- The high-frequency noise rejection is tested by injecting a high-frequency signal at the input and measuring the signals after filtering. (This test will be performed on a subset of the PACMAN cards, to validate the design)
- Temperature dependence (TBD)
- Limits of digital I/O (clock speed, long cables) (TBD)

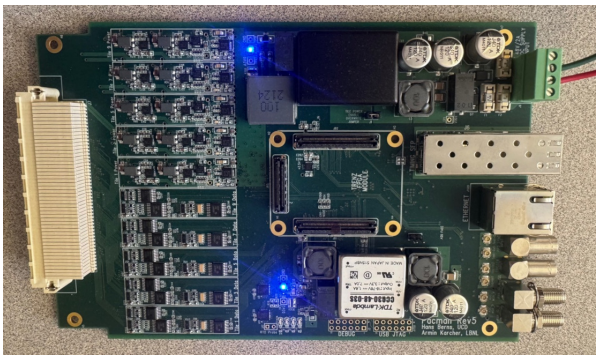
7 Hardware Checkout Procedure

Caveats: No allowed tolerances have been provided yet, because we have only tested a few boards.

7.1 Bare Board



Board Power Only



Tile Power Enabled (jumper)

The hardware checkout begins with a bare board (no TRENZ module installed). The 3.3V digital power (used by PACMAN) is on as soon as the board is powered at the DC supply input. The board also includes a jumper "TILE POWER ENABLE OVERRIDE JUMPER" which allows the 3.3 V tile power to be validated before installing any module.

The current should be measured and compared to the nominal expected value for the applied voltage, in the range of positive 32 V to positive 48 V):

Voltage (V)	Current (mA) (No Jumper)	Current (mA) (Analog Power Enabled)
48	66	100
40	77	113
36	91	131

Using a DMM, verify the supply of 3.3 V (nominal) at TP3 (D3V3) relative to TP4 (GND).

Using a DMM, verify the supply of 3.0 V (nominal) at TP13 (A3V0) relative to TP14 (GND).

7.2 TRENZ Module and Linux System Checks

With a TRENZ module installed on the board, insert an SD card with the latest hardware checkout firmware, the ethernet cable connected to the network, and power the board.

The local network may require customizations to the ROOT filesystem in order to bring up the ethernet connection. Before attempting to bring up the network on a headless PACMAN card (no USB JTAG/UART header has been installed) consider debugging the network configuration on a PACMAN equipped with a USB JTAG/UART header or a Trenz Module ... Once the network is brought up reliably, you can boot a PACMAN with no terminal and connect over ethernet.

Login to the pacman card via ethernet.

Run the following Linux diagnostic tools (TBD).

The Linux-based drivers depend the Linux GPIO interface in /sys/class. This should be initialized at boot via the gpio_init script included with the hwutil package. Confirm that this is working properly with:

```
root@Trenz:~# ls /sys/class/gpio/
export gpio906 gpio913 gpio918 gpio919 gpiochip906 unexport
```

7.3 Linux-based Hardware Checkout Application

The PACMAN Zipline firmware includes a menu interface to the Linux drivers: pacman_menu. It is available from the command line as:

```
root@Trenz:~# pacman_menu
pacman_menu: PACMAN Linux driver access via menu, for diagnostics and hardware checkout.
INFO: Opening /dev/mem.
INFO: Initializing PACMAN AXI-Lite interface.
INFO: Running pacman firmware version 3.2 (Build: 0x7fff0000 HW Code: 0x10500)
INFO: Opening /dev/mem.
INFO: Initializing PACMAN BRAM (AXI-LITE) interface.
INFO: Opening /dev/mem.
INFO: Initializing DMA contol interface (AXIL).
INFO: Initializing DMA TX_BUFFER.
INFO: Initializing DMA RX_BUFFER.
MAIN MENU: choose an option:
(1) blink LEDs (2) global registers (3) toggle scratch registers
(4) power menu (5) RX/TX menu (6) timing menu (7) ADC menu
```

The user selects menu options by typing the number for the option followed by enter. The numbers and arrangement of the menus will vary with firmware version as we further develop the tools. This documentation is based on Zipline firmware version 3.6. In the future, we will support a turn-key hardware checkout routine as a menu option. However, here we document each individual test one-by-one.

7.3.1 LED checkout

The pacman_menu can be used to blink the LEDs as follows:

```
MAIN MENU: choose an option:
(1) blink LEDs (2) global registers (3) toggle scratch registers
(4) power menu (5) RX/TX menu (6) timing menu (7) ADC menu
1
INFO: selected 1
INFO: starting LED blink test...
Blinking RED LED on Trenz Module, via MIO...
Blinking PACMAN LED-0, via MIO...
Blinking PACMAN LED-1, via MIO...
Blinking PACMAN LED-2, via AXIL register...
Blinking PACMAN LED-3, via AXIL register...
INFO: done with LED blink test.
MAIN MENU: choose an option:
(1) blink LEDs (2) global registers (3) toggle scratch registers
(4) power menu (5) RX/TX menu (6) timing menu (7) ADC menu
```

The user should verify that:

- The red LED on the TRENZ module blinks
- The four green LEDs on the PACMAN card blink one-by-one.

This simple test also confirms that the MIO and AXI-Lite register interfaces are both working.

7.3.2 Board Power

Control and monitoring of electrical power provided by the PACMAN is accessible from the power menu:

```
MAIN MENU: choose an option:
(1) blink LEDs (2) global registers (3) toggle scratch registers
(4) power menu (5) RX/TX menu (6) timing menu (7) ADC menu
4
INFO: selected 4
POWER MENU: choose an option:
(0) main menu (1) toggle enables (2) toggle power (3) monitor power
(4) write IV curves to file
```

The "toggle enables" feature steps through various combinations of the Tile Power Enable[5] and the ten individual Tile enables. The accessible settings include:

Setting	Explanation
0x00000000	Nothing Enabled
0x00010000	Tile Power enabled, no individual Tiles enabled ll
0x00010001	Tile Power enabled, only Tile 1 enabled
0x000103FF	Tile Power enabled, all Tiles enabled

The "monitor power" feature access the on-board voltage and current monitoring ADCs. At the moment, we are only concerned with the Board Power monitoring:

```
BOARD POWER SUMMARY:
Board 3V6: voltage: 3526 mV current: 1337 mA
Board 3V3: voltage: 3326 mV current: 813 mA
Board 3V0: voltage: 2985 mV current: 0 mA
```

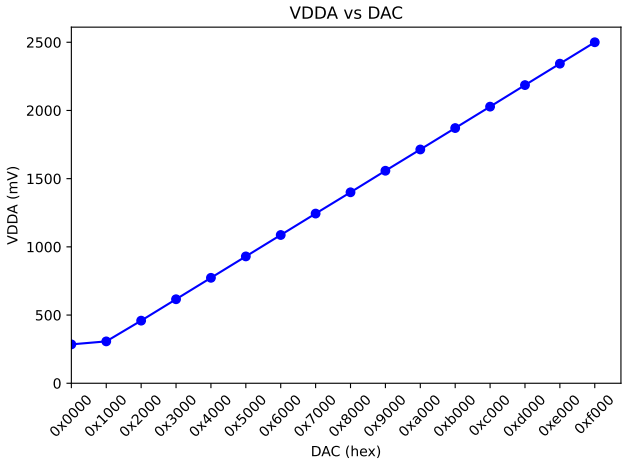
The user should verify that:

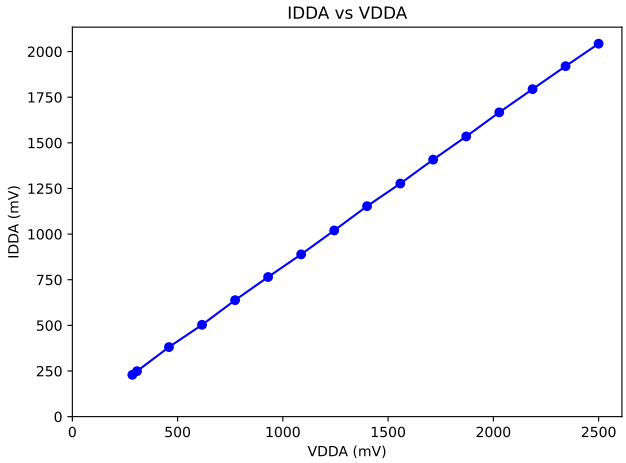
- As you step through the enables, first the Tile Power Enabled LED lights, then the Tile 1 enable LED lights, then all the Tile enable LEDs light.
- The Board 3V0 voltage is near either 3000 mV or zero depending on whether the TILE Power is enabled or not.
- (The RTD Probe feature can be tested here as well by connecting a resistor at J17)

7.3.3 VDDD and VDDA

The "write IV curves to file" feature of the Power submenu steps through several different DAC settings and steps through the monitored values of the voltages VDDA and VDDD along with the corresponding currents IDDA and IDDD. The collected data (saved as a text file) can be plotted to show the linearity of the provided voltage with DAC setting, and the IV curve for each supplied voltage.

[5]Also called "Analog Power Enable" but this is misleading as it provides both digital and analog power to the tile cards.





Here the VDDA has been loaded with a resistor to ground at the loopback card.

7.3.4 POSI/PISO Loopback Testing with Random Numbers

One of the most important checks is the RX and TX capabilities for the POSI and PISO signals between the PACMAN and the LArPix ASICs. We test these with loopback at twice the nominal ASIC clock rate. We transmit known random values out of the PACMAN, physically loop them back in as inputs. We check each received word against the random value that was sent. This test is available through the RX/TX submenu, but requires a few steps:

- Set the global RX configuration to 0x00000001. This disables heartbeat, sync, and trigger words which could confuse the test. It also only includes a single cycle in each transmitted DMA buffer.
- Set RX configuration to 0x00001001. This sets the mode to 1 (normal operation) and full speed (clock divisor 1).
- Set TX configuration to 0x00001601. This sets the mode to 1 (normal operation), phase to 0x60, and full speed (clock divisor 1).
- Stop pacman_server (both the data server and command server) via:

  ```
  root@Trenz:~# pacman_server stop
  Stopping command server...
  Stopping dataserver...
  ```

  It is most convenient if you wait to do this until after the global RX configuration has been set to disables heartbeat, sync, and trigger words. That way, the pacman_cmdserver drains the FIFO.

- Stopping pacman_cmdserver leaves a pending DMA read request, so either reset DMA, or sent a single RX. When ready for the test, a single TX leaves the fifo size at 37 (40 + 1 128 bit words minus four words buffered by the vendor supplied DMA engine) which should be read by a single RX, leaving the fifo size at 0.

Once preparations are complete, the test is easily run:

```
RX/TX MENU: choose an option:
(0) main menu (1) zero counts (2) toggle TX config (3) toggle RX config (4) toggle RX globa
(5) TX status (6) TX look (7) single TX
(8) RX status (9) RX look (10) single RX
(11) benchmark TX (12) benchmark RX/TX loopback (13) random RX/TX loopback
(14) DMA status (15) reset DMA
(16) set DMA TX to RUN
(17) set DMA RX to RUN (18) clear DMA RX (19) start DMA RX (20) resume RX
13
INFO: selected 13
INFO: Setting DMA TX To RUN.
INFO: DMA TX has left the HALTED state successfully. (timeout=100)
INFO: Setting DMA RX To RUN.
INFO: DMA RX has left the HALTED state successfully. (timeout=100)
SUMMARY: Found 0 discrepancies in 40000 uart packets
```

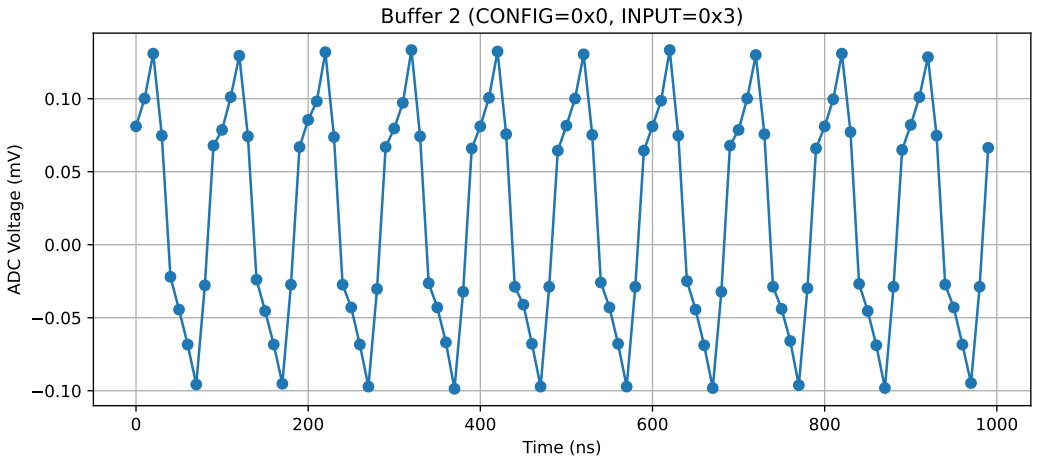
The user should verify that:

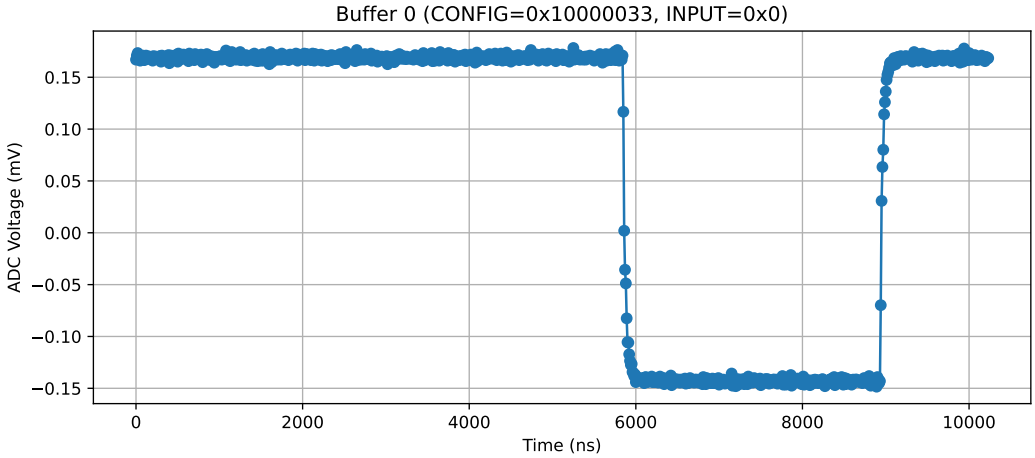
- The test can be run several times with zero discrepancies detected.

Note that to help distinguish hardware failures from software failures, you can configure internal loopback mode (so that loopback occurs in software not hardware).

7.3.5 Clock and Reset/Sync/Trigger Signals

To test the Clock and Reset/Sync/Trigger signals, these are routed back to PACMAN via loopback into the analog monitoring signals. These can be directed to the ADC, which acts as a digital oscilloscope to record the waveforms:





7.3.6 Front Panel Checkout

- (the MUX can be adjusted to provide a 100 Ohm resistance, or open circuit, between the two SMA connectors, which can be verified with a DMM.)
- (the TTL pulses can be fed into the LEMO connectors, which are counted by the timing unit, with counts available from the timing sub-menu.)

A Specifications for the PACMAN Card

A.1 Overview

Each PACMAN card provides power, digital I/O, and analog test and monitoring signals to up to ten pixel tile cards. Each pixel tile card will consist of 160 ASICs (CONFIRM).

A.2 Power

The PACMAN card provides digital and analog power to up to ten pixel tile cards. The digital voltage level (VDDD) and analog voltage level (VDDA) are digitally controlled independently for each pixel tile card, within the limits and at the stability specified below.

The power requirements of the PACMAN are based on scaling the measured power consumption of the 100 ASIC pixel tile cards to 160 ASIC pixel tile cards of the production system, a scale factor of 1.6. An additional (minimum) safety factor of 10% is added to these extrapolated power requirements to derive the following specifications:

Table 14: Digital and Analog Power Requirements (per tile card)

	Stability	Min (V)	Max (V)	Max Current (A)	Max Power (W)
VDDA	1%	0	1.8	0.3	0.7
VDDD	1%	0	1.8	1.1	2.2

Reaching a 1% stability on VDDA and VDDD requires linear regulation and so the power requirements at the input of the PACMAN are derived from a higher voltage level of 3 V. This implies that the initial switching voltage requirement must provide at least 42 W at 3 V.

The PACMAN power is provided by a single nominal 48 V 2 A DC input. The power connection is the four position terminal connector (Phoenix Contact P/N 5444660). The inner two pins provide power and the outer two are for sense:

Table 15: Input Power Connections

pin	function
1	V+ sense
2	V+
3	V-
4	V- sense

The connector is rated for 8 A and 300 V. The PACMAN is fused at 2.5 A and has over voltage protection at 60 V. The sense pins are fused at 0.5 A.

A.3 Digital Signals

The LArPix-v2 ASIC uses a bit-serial protocol to transmit and receive data. The PACMAN is the primary and the LARPix ASIC the secondary, so that the PACMAN transmits POSI and receives PISO signals.

The PACMAN shall provide the following digital IO signals to each pixel tile card:

Table 16: Digital Signals (per pixel tile card)

Signal	Quantity	Description
POSI	4	Bit serial transmission to pixel tile
PISO	4	Bit serial reception from pixel tile
CLK	1	Clock
TRIG	1	Trigger
SYNC	1	Multiplexed Synchronization and Reset

Each POSI signal is received by a single ASIC, but the clock, trigger, and sync signals are fanned out to every ASIC on the tile card. The PACMAN must be capable of driving the resulting capacitive load.

The nominal clock rate is 10 MHz with a maximum of 40 MHz. The LArPix ASIC uses an LVDS signal with custom voltage level which the PACMAN is capable of transmitting and receiving.

A.4 Noise

The noise injected into the pixel tile from the controller is less than 1 mV.

A.5 Pixel Tile Interface

A single PACMAN card drive up to ten pixel tile cards through a single flange card. The interface to the eight-tile flange card is through a 6 row by 50 pin high-density SAMTEC connector `SEAF-50-01-L-06-1-RA-K-TR` on the PACMAN card.

The PACMAN card shall provide a footprint for an optional 2x17 connector to drive a pixel card directly without the use of flange card.

A.6 Front Panel Interface

The PACMAN card implements the front-panel interfaces listed below in the specified quantity with specified routing. When routing to programmable logic (PL) is specified, the specified purpose is nominal and can be repurposed through firmware changes.

Table 17: Required Front Panel Interface

Interface	Quantity	Routing
SMA Jack	1	Multiplexed to eight tile analog monitor
SMA Jack	1	Multiplexed to eight tile adc test
Lemo Jack	2	PL: Trigger and Sync
Push Button	1	Reset
LED	4	PL: General Purpose

A.7 Timing System Endpoint

The PACMAN card implement a ProtoDUNE-SP timing system endpoint[6] of a single fiber-optic link.

A.8 Embedded Linux

The PACMAN is an embedded Linux system and supports several standard embedded Linux interfaces:

Interface	P/N	Notes
Ethernet		Gigabit
SD Card		Bootable from SD card
JTAG		SoC Configuration
UART		Linux Terminal

The JTAG and UART interface are multiplexed to provide a linux terminal and firmware configuration over a single USB port.

[6] DUNE-doc-1651-v3

B PACMAN Version History

Table 18: PACMAN Evolution

Card	Designer	Date	Notes
PACMAN 1v1	Hillbrand	Feb 2020	Initial Prototype (Not Supported)
PACMAN 1v2	Karcher	July 2020	Single-tile capable (Not Supported)
PACMAN 1v3	Berns/Karcher	Jan 2021	Eight-tile capable - Module 0
PACMAN 1v4	Berns/Karcher	Feb 2022	LArPix-v2b, Power, TX
PACMAN 1v5	Berns/Karcher	Aug 2024	Ten-tile capable
PACMAN 1v5b	Berns/Karcher	TBD 2025	Interchangeable w/ 1v5

C Prototype Results

Many of the specifications for the PACMAN are driven by measurements, using prototype devices, as described in this section.

The Intrinsic RMS noise on the LArPix ASIC was benchtested[7] at ~ 3 mV.

The PACMAN card provides digital (VDDD) and analog (VDDA) power for the LArPix ASICs. The ASIC power draw during power up was measured[8] as follows:

Table 19: ASIC Current Draw at Power-up

	Voltage (V)	Current (mA)	Power (mW)	Power (μW/chan)
VDDA	1.8	1.4	2.5	39
VDDD	1.8	6.92	12.5	195

[7] Reported 6 Mar 2020, Brooke

[8] Reported 13 Feb 2020, Brooke

The 10x10 pixel-tile power draw was measured [9] as follows:

Table 20: 10x10 Power Draw

	Voltage (V)	Current (mA)	Power (W)
VDDA	1.8	166	0.3
VDDD	1.8	611	1.1

[9] Reported 14 Jan 2021, Brooke